DEVPILOT AI

COMPLETE DATABASE ARCHITECTURE & ERD

UPDATED WITH PLANS, SUBSCRIPTIONS, AI USAGE, CREDITS, RATE LIMITING, AUDITING & SOFT DELETE

Architecture and database design reference

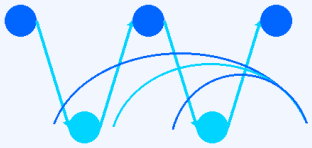
SYSTEM DESIGN

DevPilot AI

Complete Database Architecture & ERD

Identity · Debugging · Knowledge · GitHub · AI Usage · Plans · Credits · Rate Limiting · Auditing

ARCHITECTURE REFERENCE



DATABASE **Microsoft SQL Server**	BACKEND **ASP.NET Core Web API**
ARCHITECTURE **Clean Architecture**	DOCUMENT STATUS **Architecture / Database Design Reference**

CONTENTS & DOCUMENT GUIDE

This reference retains the complete source material while presenting major relationships and flows as diagrams. The entity-by-entity sections remain the detailed source for fields, relationships, lifecycle semantics, and security rules.

01 DOCUMENT PURPOSE
02 ARCHITECTURAL PRINCIPLES
03 STATUS OF THE CURRENT ARCHITECTURE
04 COMPLETE TABLE CATALOG
05 COMMON TIMESTAMP AND SOFT DELETE CONVENTIONS
06 TABLE 01 - Users
07 TABLE 02 - Roles
08 TABLE 03 - UserRoles
09 TABLE 04 - EmailVerifications
10 TABLE 05 - RefreshTokens
11 TABLE 06 - Problems
12 TABLE 07 - ProblemRequests
13 TABLE 08 - ProblemAttachments
14 TABLE 09 - DebuggingAttempts
15 TABLE 10 - DeveloperFeedback
16 TABLE 11 - ProblemSolutions
17 TABLE 12 - KnowledgeArticles
18 TABLE 13 - ArticleVersions
19 TABLE 14 - Technologies
20 TABLE 15 - ProblemTechnologies
21 TABLE 16 - Challenges
22 TABLE 17 - ChallengeAttempts
23 TABLE 18 - ChallengeEvaluations
24 TABLE 19 - GitHubAccounts
25 TABLE 20 - GitHubRepositories
26 TABLE 21 - CodeReviews
27 TABLE 22 - CodeReviewFindings
28 TABLE 23 - AIProviders
29 TABLE 24 - AIModels
30 TABLE 25 - AIRequests
31 TABLE 26 - ProblemEmbeddings
32 TABLE 27 - SimilarityMatches
33 TABLE 28 - Plans
34 TABLE 29 - Features
35 TABLE 30 - PlanFeatures
36 TABLE 31 - UserSubscriptions
37 TABLE 32 - AIUsageEvents
38 TABLE 33 - UsagePeriods
39 TABLE 34 - UsageLedger
40 TABLE 35 - AuditLogs
41 TABLE 36 - RateLimitLogs (OPTIONAL)
42 PLAN MANAGEMENT RULES
43 FEATURE ENTITLEMENT RULES
44 AI QUOTA AND CREDIT RULES
45 AI REQUEST VS AI USAGE EVENT
46 AI OPERATION TYPES
47 TOKEN ACCOUNTING
48 COST ACCOUNTING
49 RATE LIMITING ARCHITECTURE
50 RATE LIMIT PARTITIONING
51 RATE LIMIT RESPONSE
52 RATE LIMIT VS AI QUOTA
53 AI OPERATION END-TO-END FLOW
54 CONCURRENCY CONTROL
55 IDEMPOTENCY
56 SOFT DELETE ARCHITECTURE
57 USER ACCOUNT SOFT DELETE FLOW
58 KNOWLEDGE RETENTION AFTER ACCOUNT DELETION
59 QUERY FILTER RULES
60 UNIQUE INDEX STRATEGY
61 TRANSACTIONAL OPERATIONS
62 ADMIN CAPABILITIES
63 USER USAGE CAPABILITIES
64 USAGE ANALYTICS
65 AI GATEWAY RELATIONSHIP WITH ACCOUNTING
66 TOKEN OPTIMIZATION DATA FLOW
67 GITHUB AND AI USAGE
68 TELEGRAM AND AI USAGE
69 APPEND-ONLY DATA
70 PLAN / USAGE SOFT DELETE RULE
71 DATABASE INDEXING RECOMMENDATIONS
72 DATA INTEGRITY RULES
73 SECURITY RULES
74 ERROR AND FAILURE ACCOUNTING
75 ADMIN PLAN CHANGE AUDITING
76 FUTURE PAYMENT SUPPORT
77 FUTURE DISTRIBUTED RATE LIMITING
78 TABLE LIFECYCLE CLASSIFICATION SUMMARY
79 MASTER RELATIONSHIP MAP
80 COMPLETE BUSINESS FLOW WITH THE UPDATED MODEL
81 ACCOUNT DELETION WITH THE UPDATED PLAN/USAGE MODEL
82 DO NOT CONFUSE THESE CONCEPTS
83 IMPLEMENTATION ORDER FOR THE NEW SUBSYSTEM
84 EXPECTED DATABASE SAFETY TESTS
85 FINAL ARCHITECTURAL DECISION
86 IMPORTANT IMPLEMENTATION NOTES

Core design themes: history preservation · provider independence · transactional credit accounting · native request-rate protection · selective soft delete · no cascade delete

DOCUMENT CONTEXT & ARCHITECTURE AT A GLANCE

Scope and continuity

This document preserves the original DevPilot AI database model and extends it with the agreed Plan Management, User Subscription, AI Usage, AI Credits, Rate Limiting, Audit, and Soft Delete architecture. It does NOT replace the original system model with a different application. It also does NOT add entities merely for symmetry. Each entity is classified according to its actual business meaning.

Project: DevPilot AI | Database: Microsoft SQL Server | Backend: ASP.NET Core Web API | Architecture: Clean Architecture

Declared architecture sequence: Presentation / API → Application → Domain → Infrastructure → Database

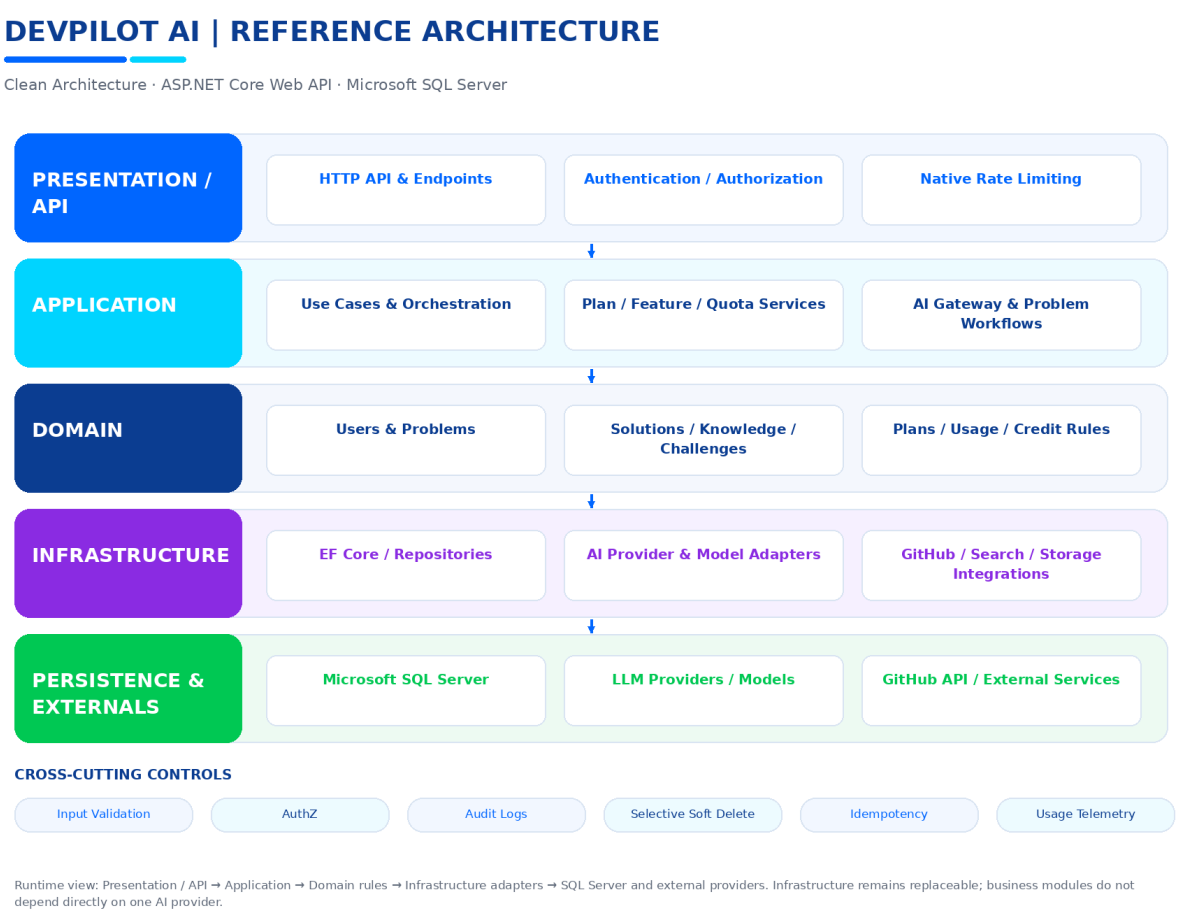


Figure 0. DevPilot AI reference architecture and cross-cutting controls.

Separation of concerns

Rate limits control how frequently requests arrive. Usage quotas control cumulative consumption over a period. They are separate mechanisms and must be enforced independently.

01 DOCUMENT PURPOSE

This document explains the complete database architecture of DevPilot AI from the beginning, including:

1. Identity and authentication.
2. Problems and AI debugging.
3. Knowledge Base and article evolution.
4. Challenges and developer practice.
5. GitHub integration and code review.
6. AI providers, models, requests, and similarity search.
7. Plan Management.
8. User Subscriptions.
9. Plan Features and feature entitlements.
10. AI usage tracking.
11. AI token and cost accounting.
12. AI Credits and quota periods.
13. Usage ledger.
14. Built-in API rate limiting.
15. Security and administrative audit logs.
16. Soft Delete.
17. Relationships, constraints, indexes, transactions, and lifecycle rules.

The original DevPilot AI database is designed around the complete lifecycle of **a real developer problem:**

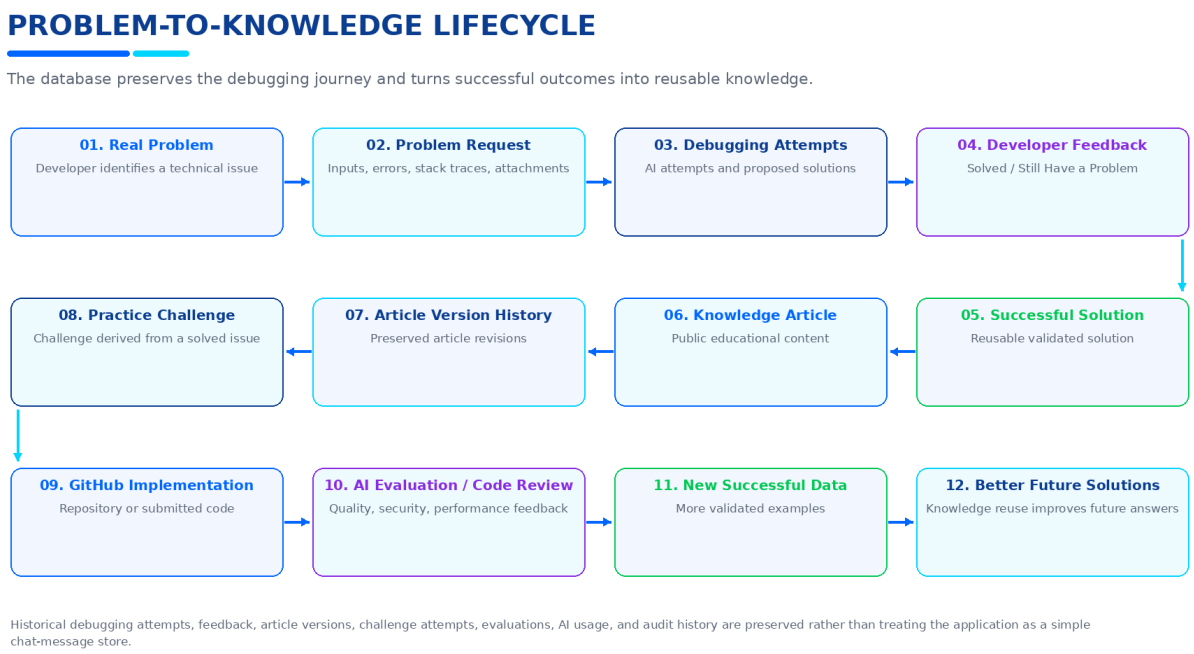


Figure 1. Problem-to-knowledge lifecycle.

The database must preserve this history rather than behaving like a simple chat message store.

02 ARCHITECTURAL PRINCIPLES

2.1 Developer-Centered

The User is the primary owner of personal application activity.

A User can own or initiate:

- Problems
- Challenge Attempts
- GitHub Accounts
- Code Reviews
- AI Requests

- Subscriptions
- Usage records

2.2 Problem-Centered

Problem is the central business object for debugging.

A Problem can connect to:

- Problem Requests
- Attachments
- Debugging Attempts
- Feedback
- Successful Solutions
- Knowledge Articles
- Technologies
- Challenges
- AI Requests
- Similarity Matches

2.3 History Preservation

DevPilot AI intentionally preserves:

- Previous debugging attempts
- Successful solutions
- Article versions
- Challenge attempts
- Evaluations
- AI usage information
- Audit history

2.4 Knowledge Reuse

Solved problems become reusable knowledge.

Similar future problems can reuse:

- Successful Solutions
- Knowledge Articles
- Similarity Matches
- Relevant debugging history

2.5 AI Provider Independence

Application business modules do not depend directly on one LLM provider.

The AI Gateway selects:

- Provider
- Model
- Priority
- Availability
- Fallback provider/model

Usage accounting remains independent from provider selection.

2.6 Separation of Rate Limits and Quotas

RATE LIMIT:

Controls request frequency over short windows.

Examples:

- 5 AI requests per minute
- 10 follow-up requests per minute
- 5 login attempts per 5 minutes

USAGE QUOTA:

Controls cumulative consumption over a usage period.

Examples:

- 50 AI credits per month
- Maximum analyses in a period
- Feature-specific allowances

Rate limiting is NOT the usage accounting system.

2.7 No Cascade Delete

This project has a strict NO CASCADE policy.

Database foreign keys must not use:

ON DELETE CASCADE

EF Core relationships must use a non-cascading behavior, with DeleteBehavior.NoAction preferred where appropriate.

Soft Delete is implemented at the application/data-query level and must not be implemented using database cascade deletion.

2.8 Soft Delete Is Selective

Soft Delete is NOT applied to every table.

It is used only when the entity has meaningful "deleted" semantics and the row should remain in the database.

Entities with their own lifecycle (for example RefreshToken, AIModel, EmailVerification, UserSubscription) must use their dedicated state model instead of an unnecessary IsDeleted flag.

03 STATUS OF THE CURRENT ARCHITECTURE

CURRENT IMPLEMENTED FOUNDATIONS:

- Users
- Roles
- UserRoles
- EmailVerifications
- RefreshTokens
- Problems
- ProblemRequests
- ProblemAttachments
- DebuggingAttempts
- DeveloperFeedback
- ProblemSolutions
- KnowledgeArticles
- ArticleVersions
- Technologies
- ProblemTechnologies
- Challenges
- ChallengeAttempts
- ChallengeEvaluations
- GitHubAccounts
- GitHubRepositories
- CodeReviews
- CodeReviewFindings
- AIProviders
- AIModels
- AIRequests
- ProblemEmbeddings
- SimilarityMatches

CURRENT SOFT DELETE IMPLEMENTATION:

- User
- Problem
- ProblemAttachment
- ProblemSolution
- KnowledgeArticle
- Challenge
- GitHubRepository
- CodeReview

CURRENT SOFT DELETE MECHANISM:

- ISoftDeletable
- IsDeleted
- DeletedAt
- EF Core global query filters
- DELETE /api/users/me
- JWT validation checks active User state
- Refresh token revocation on account deletion

- No Cascade Delete

PLANNED ARCHITECTURE TO PREPARE NOW:

- Plans
- Features
- PlanFeatures
- UserSubscriptions
- AIUsageEvents
- UsagePeriods
- UsageLedger
- AuditLogs
- Optional RateLimitEvents / RateLimitLogs for operational telemetry

IMPORTANT:

The Plan, Usage, Credit, Subscription, and Audit entities in this document are architectural preparation. They must only be considered "implemented" after code, migration, database, and tests have actually been completed.

04 COMPLETE TABLE CATALOG

HIGH-LEVEL DATA DOMAIN MAP

Domain grouping for the 36-table catalog; detailed fields and relationships are documented in the entity sections.

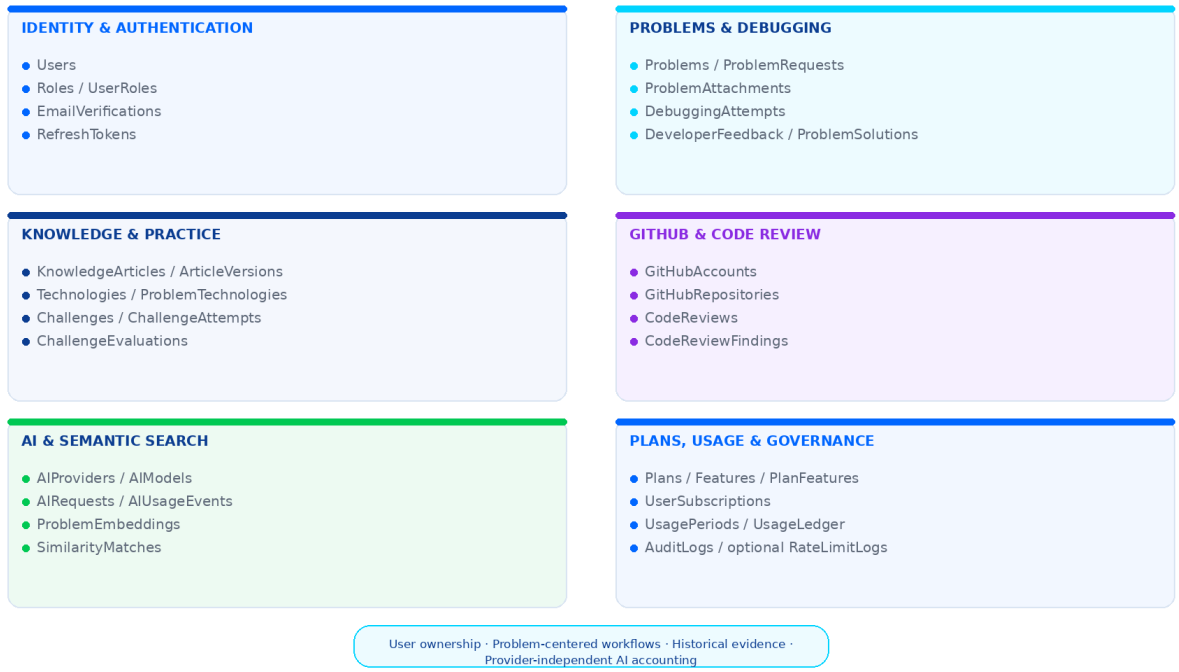


Figure. High-level data domain map for the complete table catalog.

IDENTITY & AUTHENTICATION
01. Users
02. Roles
03. UserRoles
04. EmailVerifications
05. RefreshTokens
PROBLEMS & DEBUGGING
06. Problems
07. ProblemRequests
08. ProblemAttachments
09. DebuggingAttempts
10. DeveloperFeedback
11. ProblemSolutions
KNOWLEDGE
12. KnowledgeArticles
13. ArticleVersions
14. Technologies
15. ProblemTechnologies
PRACTICE
16. Challenges
17. ChallengeAttempts
18. ChallengeEvaluations
GITHUB & CODE REVIEW
19. GitHubAccounts
20. GitHubRepositories
21. CodeReviews
22. CodeReviewFindings
AI & SEARCH
23. AIProviders

24. AIModels
25. AIRequests
26. ProblemEmbeddings
27. SimilarityMatches

PLANS & SUBSCRIPTIONS

28. Plans
29. Features
30. PlanFeatures
31. UserSubscriptions

AI USAGE & CREDIT ACCOUNTING

32. AIUsageEvents
33. UsagePeriods
34. UsageLedger

SECURITY / OPERATIONS

35. AuditLogs

OPTIONAL OPERATIONS TELEMETRY

36. RateLimitLogs

NOTE:
RateLimitLogs is optional. Native ASP.NET Core rate limiting does not require a database table. Use it only when durable operational telemetry is needed. Technical limiter state remains the responsibility of the rate limiter.

05 COMMON TIMESTAMP AND SOFT DELETE CONVENTIONS

Common timestamp fields where applicable:

CreatedAt

Date/time the row was created.

UpdatedAt

Date/time the row was last updated.

DeletedAt

Date/time the row was logically deleted.

Soft-deletable entities use:

IsDeleted BIT NOT NULL DEFAULT 0

DeletedAt DATETIME2 NULL

Existing records after migration:

IsDeleted = 0

DeletedAt = NULL

Global Query Filter behavior:

Normal queries automatically exclude IsDeleted = true.

Deleted data may only be queried intentionally using an internal/admin mechanism such as IgnoreQueryFilters(), subject to authorization.

IMPORTANT:

IsDeleted is NOT automatically added to:

- RefreshTokens
- EmailVerifications
- AuditLogs
- AIUsageEvents
- UsageLedger
- Roles
- entities whose lifecycle is better represented by an existing state field.

06 TABLE 01 - Users

Purpose:

Stores developer accounts.

Main fields:

- UserId INT PK
- Name NVARCHAR(150)
- Email NVARCHAR(255) UNIQUE
- PasswordHash NVARCHAR(500)
- EmailVerifiedAt DATETIME2 NULL
- CreatedAt DATETIME2
- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- Users 1:N UserRoles
- Users 1:N EmailVerifications
- Users 1:N RefreshTokens
- Users 1:N Problems
- Users 1:N ChallengeAttempts
- Users 1:N GitHubAccounts
- Users 1:N CodeReviews
- Users 1:N AIRequests
- Users 1:N UserSubscriptions
- Users 1:N AIUsageEvents
- Users 1:N UsagePeriods
- Users 1:N UsageLedger entries
- Users 1:N AuditLogs

Soft Delete:

YES.

Delete behavior:

- IsDeleted = true
- DeletedAt = UTC timestamp
- Active refresh tokens are revoked
- Existing history remains
- Login is blocked
- Refresh is blocked
- Protected APIs reject deleted users
- Password reset is blocked
- Email verification workflows are blocked

Email uniqueness:

The existing unique Users.Email rule should remain unless a future explicit product decision changes identity recycling behavior.
Account deletion is NOT a physical row deletion.

07 TABLE 02 - Roles

Purpose:

Stores authorization roles.

Main fields:

- RoleId INT PK
- Name NVARCHAR(50) UNIQUE

Initial roles:

- Developer
- Administrator

Relationships:

- Roles 1:N UserRoles

Soft Delete:

NO.

Reason:

Roles are system/reference data. Mandatory roles must remain available.

Lifecycle:

Roles are managed as system configuration rather than user-owned records.

08 TABLE 03 - UserRoles

Purpose:

Junction table assigning roles to users.

Main fields:

- UserId INT PK/FK
- RoleId INT PK/FK

Primary key:

(UserId, RoleId)

Relationships:

- Users 1:N UserRoles
- Roles 1:N UserRoles

Soft Delete:

NO by default.

Reason:

This is a relationship table. The relationship itself does not require deletion semantics in the current authentication design.
Relationship changes must never result in cascade deletion of users or roles.

09 TABLE 04 - EmailVerifications

Purpose:

Stores email verification workflows.

Main fields:

- VerificationId INT PK

- UserId INT FK
- CodeHash NVARCHAR(500)
- TokenHash NVARCHAR(500) NULL
- Method VARCHAR(20)
- ExpiresAt DATETIME2
- Attempts INT
- VerifiedAt DATETIME2 NULL
- CreatedAt DATETIME2

Relationships:

- Users 1:N EmailVerifications

Soft Delete:

NO.

Lifecycle:

EmailVerifications have a verification-specific state:

- Pending
- Verified
- Expired
- Invalidated by a newer verification

Security:

- Store hashes, not raw OTP/token values.
- Apply expiration.
- Apply maximum attempts.
- Apply resend cooldown.
- Deleted accounts cannot verify or resend verification.

Historical verification rows may remain for security and audit purposes.

10 TABLE 05 - RefreshTokens

Purpose:

Stores refresh-token sessions.

Main fields:

- RefreshTokenId INT PK
- UserId INT FK
- TokenHash NVARCHAR(255)
- ExpiresAt DATETIME2
- IsRevoked BIT
- RevokedAt DATETIME2 NULL
- IsUsed BIT
- ReplacedByTokenHash NVARCHAR(MAX) NULL
- CreatedAt DATETIME2

Relationships:

- Users 1:N RefreshTokens

Soft Delete:

NO.

Reason:

RefreshTokens already have a purpose-specific lifecycle.

Security lifecycle:

- Valid
- Used
- Revoked
- Expired
- Replaced

Account deletion:

- Revoke all active tokens
- Do not physically delete token history

Refresh token reuse detection remains independent from Soft Delete.

11 TABLE 06 - Problems

Purpose:
Represents a developer's technical problem.

Main fields:

- ProblemId INT PK
- UserId INT FK
- ProblemCode VARCHAR(20) UNIQUE
- Title NVARCHAR(250)
- Description NVARCHAR(MAX)
- Status VARCHAR(30)
- CreatedAt DATETIME2
- SolvedAt DATETIME2 NULL
- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- Users 1:N Problems
- Problems 1:N ProblemRequests
- Problems 1:N DebuggingAttempts
- Problems 1:N ProblemSolutions
- Problems 1:1 KnowledgeArticles
- Problems 1:N ProblemTechnologies
- Problems 1:N Challenges
- Problems 1:N AIRequests
- Problems 1:N SimilarityMatches

Soft Delete:
YES.

Meaning:
A developer may remove a problem from active views without destroying its debugging history or knowledge value.

Normal queries:
Exclude soft-deleted Problems.

Internal/admin queries:
May intentionally include deleted Problems.

12 TABLE 07 - ProblemRequests

Purpose:
Stores every user request/input related to a Problem.

Main fields:

- RequestId INT PK
- ProblemId INT FK
- Content NVARCHAR(MAX)
- ErrorMessage NVARCHAR(MAX) NULL
- StackTrace NVARCHAR(MAX) NULL
- TechnologyHint NVARCHAR(200) NULL
- CreatedAt DATETIME2

Relationships:

- Problems 1:N ProblemRequests
- ProblemRequests 1:N ProblemAttachments
- ProblemRequests 1:N DebuggingAttempts
- ProblemRequests 1:1 ProblemEmbeddings
- ProblemRequests 1:N SimilarityMatches

Soft Delete:
NO.

Reason:
ProblemRequests form part of the debugging history and are intentionally preserved.
A deleted Problem may still have its historical requests retained.

13 TABLE 08 - ProblemAttachments

Purpose:
Stores files attached to problem requests.

Main fields:

- AttachmentId INT PK
- RequestId INT FK
- FileType VARCHAR(30)
- FileName NVARCHAR(255)
- StoragePath NVARCHAR(500)
- CreatedAt DATETIME2
- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- ProblemRequests 1:N ProblemAttachments

Soft Delete:
YES.

Meaning:
A developer may logically remove an attachment while retaining the record and its audit/security reference.

Important:
Database Soft Delete does not automatically mean physical file destruction. File-storage retention/deletion policy must be handled by the application storage layer.

14 TABLE 09 - DebuggingAttempts

Purpose:
Stores every AI debugging attempt.

Main fields:

- AttemptId INT PK
- ProblemId INT FK
- RequestId INT FK
- AttemptNumber INT
- RootCause NVARCHAR(MAX)
- SolutionText NVARCHAR(MAX)
- WhyItWorks NVARCHAR(MAX)
- CreatedAt DATETIME2

Relationships:

- Problems 1:N DebuggingAttempts
- ProblemRequests 1:N DebuggingAttempts
- DebuggingAttempts 1:1 DeveloperFeedback
- DebuggingAttempts 1:N ProblemSolutions

Soft Delete:
NO.

Reason:
This is historical debugging evidence.

Previous attempts must remain available for:

- Debugging history
- Knowledge generation
- Quality analysis
- Successful solution evaluation

15 TABLE 10 - DeveloperFeedback

Purpose:
Stores developer feedback after a debugging attempt.

Main fields:

- FeedbackId INT PK
- AttemptId INT FK

- Result VARCHAR(30)
- Comment NVARCHAR(MAX) NULL
- CreatedAt DATETIME2

Allowed results:

- Solved
- StillHaveAProblem

Relationships:

- DebuggingAttempts 1:1 DeveloperFeedback

Soft Delete:
NO.

Reason:
Feedback is historical evidence that determines whether a proposed solution worked.

16 TABLE 11 - ProblemSolutions

Purpose:
Stores successful/reusable solutions.

Main fields:

- SolutionId INT PK
- ProblemId INT FK
- SourceAttemptId INT FK
- SolutionText NVARCHAR(MAX)
- SuccessCount INT
- LastSuccessfulAt DATETIME2
- IsPreferred BIT
- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- Problems 1:N ProblemSolutions
- DebuggingAttempts 1:N ProblemSolutions

Soft Delete:
YES.

Meaning:
A solution can be retired/deprecated without losing the historical solution.

Knowledge reuse:
Only non-deleted, eligible solutions should normally be considered for similarity reuse.

IsPreferred:
Represents preference, not deletion.

17 TABLE 12 - KnowledgeArticles

Purpose:
Represents public educational content generated from solved problems.

Main fields:

- ArticleId INT PK
- ProblemId INT FK
- Title NVARCHAR(250)
- CurrentVersionId INT NULL
- Status VARCHAR(30)
- UpdatedAt DATETIME2
- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- Problems 1:1 KnowledgeArticles
- KnowledgeArticles 1:N ArticleVersions

Soft Delete:
YES.

Meaning:
An article can be hidden/retired while preserving its history.

Public visibility must depend on:

- IsDeleted = false
- Article status
- Publication rules

A deleted article should not appear in normal public article listing.

18 TABLE 13 - ArticleVersions

Purpose:
Preserves every generated/improved version of an article.

Main fields:

- VersionId INT PK
- ArticleId INT FK
- VersionNumber INT
- ArticleContent NVARCHAR(MAX)
- UpdateReason NVARCHAR(500)
- CreatedAt DATETIME2

Relationships:

- KnowledgeArticles 1:N ArticleVersions

Soft Delete:
NO.

Reason:
Version history must remain preserved.
CurrentVersionId on KnowledgeArticles points to the currently selected version.

19 TABLE 14 - Technologies

Purpose:
Stores technology/framework taxonomy.

Main fields:

- TechnologyId INT PK
- Name NVARCHAR(100) UNIQUE

Relationships:

- Technologies 1:N ProblemTechnologies

Soft Delete:
NO by default.

Reason:
Reference data does not need ordinary user deletion.
Technology management should use controlled administrator/reference-data lifecycle semantics if required.

20 TABLE 15 - ProblemTechnologies

Purpose:
Junction between Problems and Technologies.

Main fields:

- ProblemId INT PK/FK
- TechnologyId INT PK/FK

Primary key:
(ProblemId, TechnologyId)

Relationships:

- Problems N:M Technologies

Soft Delete:
NO by default.
Relationship removal follows the current application relationship-management design.

21 TABLE 16 - Challenges

Purpose:
Stores practical coding challenges derived from solved problems.

Main fields:

- ChallengeId INT PK
- ProblemId INT FK
- Title NVARCHAR(250)
- Description NVARCHAR(MAX)
- Difficulty VARCHAR(30)
- Technology NVARCHAR(100)
- CreatedAt DATETIME2
- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- Problems 1:N Challenges
- Challenges 1:N ChallengeAttempts

Soft Delete:
YES.

Meaning:
A Challenge can be retired from the active catalog while existing attempts remain intact.

22 TABLE 17 - ChallengeAttempts

Purpose:
Stores an individual developer attempt at a Challenge.

Main fields:

- AttemptId INT PK
- ChallengeId INT FK
- UserId INT FK
- GitHubRepositoryId INT NULL FK
- Status VARCHAR(30)
- StartedAt DATETIME2
- CompletedAt DATETIME2 NULL

Relationships:

- Challenges 1:N ChallengeAttempts
- Users 1:N ChallengeAttempts
- GitHubRepositories 1:N ChallengeAttempts

Soft Delete:
NO.

Reason:
Attempt history is educational and evaluative history.

Status examples:

- Started
- Submitted
- Evaluating
- Evaluated
- Completed
- Failed

23 TABLE 18 - ChallengeEvaluations

Purpose:
Stores evaluation results of challenge attempts.

Main fields:

- EvaluationId INT PK
- AttemptId INT FK
- FunctionalScore DECIMAL(5,2)
- CodeQualityScore DECIMAL(5,2)

- SecurityScore DECIMAL(5,2)
- PerformanceScore DECIMAL(5,2)
- Feedback NVARCHAR(MAX)
- CreatedAt DATETIME2

Relationships:

- ChallengeAttempts 1:N ChallengeEvaluations

Soft Delete:
NO.

Reason:
Evaluation is historical evidence and must be preserved.

24 TABLE 19 - GitHubAccounts

Purpose:
Stores a developer's connected GitHub account and OAuth integration state.

Main fields from the approved model:

- GitHubAccountId INT PK
- UserId INT FK
- GitHubUserName NVARCHAR(100)
- AccessToken / credential reference
- ConnectedAt DATETIME2

Relationships:

- Users 1:N GitHubAccounts
- GitHubAccounts 1:N GitHubRepositories

Soft Delete:
NO as a generic rule.

Reason:
GitHub connection has its own lifecycle.

Recommended lifecycle:

- Connected
- Revoked
- Disconnected
- Reauthorized

Credentials must remain protected and must never be returned to the frontend.
Historical code review/challenge references must remain intact.

25 TABLE 20 - GitHubRepositories

Purpose:
Stores repositories imported/linked from GitHub.

Main fields:

- GitHubRepositoryId INT PK
- GitHubAccountId INT FK
- ExternalRepositoryId VARCHAR(100)
- RepositoryName NVARCHAR(200)
- OwnerName NVARCHAR(150)
- IsPrivate BIT

Current Soft Delete extension:

- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- GitHubAccounts 1:N GitHubRepositories
- GitHubRepositories 1:N ChallengeAttempts
- GitHubRepositories 1:N CodeReviews

Soft Delete:
YES.

Meaning:
A repository can be unlinked from the active workspace without destroying historical code-review/challenge references.

Private repository information remains private regardless of deletion state.

26 TABLE 21 - CodeReviews

Purpose:
Stores AI code-review sessions.

Main fields:

- CodeReviewId INT PK
- GitHubRepositoryId INT FK
- UserId INT FK
- BranchName NVARCHAR(255) NULL
- PullRequestNumber INT NULL
- Status VARCHAR(30)
- CreatedAt DATETIME2
- IsDeleted BIT NOT NULL DEFAULT 0
- DeletedAt DATETIME2 NULL

Relationships:

- Users 1:N CodeReviews
- GitHubRepositories 1:N CodeReviews
- CodeReviews 1:N CodeReviewFindings

Soft Delete:
YES.

Meaning:
A review can be dismissed/removed from active dashboards while preserving historical findings and review history.

27 TABLE 22 - CodeReviewFindings

Purpose:
Stores individual findings inside a Code Review.

Main fields:

- FindingId INT PK
- CodeReviewId INT FK
- FilePath NVARCHAR(500)
- Severity VARCHAR(30)
- Category VARCHAR(50)
- Problem NVARCHAR(MAX)
- Suggestion NVARCHAR(MAX)

Relationships:

- CodeReviews 1:N CodeReviewFindings

Soft Delete:
NO.

Reason:
Findings are historical details of a review.

28 TABLE 23 - AIProviders

Purpose:
Stores configurable LLM providers.

Main fields:

- ProviderId INT PK
- Name NVARCHAR(100)
- BaseUrl NVARCHAR(500)
- Priority INT
- IsActive BIT
- CreatedAt DATETIME2
- UpdatedAt DATETIME2

Relationships:

- AIProviders 1:N AIModels

Soft Delete:
NO by default.

Reason:
Provider activation is better represented with IsActive.

Security:
API credentials are NOT stored here in plaintext and must never be exposed to frontend clients.

29 TABLE 24 - AIModels

Purpose:
Stores models associated with AI providers.

Main fields:

- ModelId INT PK
- ProviderId INT FK
- ModelName NVARCHAR(150)
- RequestsUsed INT
- TokensUsed BIGINT
- IsAvailable BIT

Relationships:

- AIProviders 1:N AIModels
- AIModels 1:N AIRequests

Soft Delete:
NO by default.

Reason:
Model availability/state already has dedicated semantics.

IMPORTANT FUTURE DESIGN:
RequestsUsed and TokensUsed can remain as summary/cache counters, but the durable source of truth for detailed accounting should be AIUsageEvents.

30 TABLE 25 - AIRequests

Purpose:
Stores logical AI processing operations.

Main fields:

- AIRequestId BIGINT PK
- ModelId INT FK
- UserId INT NULL FK
- ProblemId INT NULL FK
- OperationType VARCHAR(50)
- RequestText NVARCHAR(MAX)
- ResponseText NVARCHAR(MAX) NULL
- TokensUsed INT NULL
- Status VARCHAR(30)
- CreatedAt DATETIME2

Relationships:

- AIModels 1:N AIRequests
- Users 1:N AIRequests
- Problems 1:N AIRequests

Soft Delete:
NO.

Reason:
AIRequests are operational/historical usage records.

Logical AI Operation:
One AIRequest represents a business-level AI operation.

Examples:

- Debugging
- Root Cause Analysis
- Article Generation

- Challenge Generation
- Challenge Evaluation
- Code Review
- Repository Analysis
- Solution Synthesis

IMPORTANT:
AIRequests should not be the only place where token consumption is stored.
Provider-level usage should be recorded through AIUsageEvents.

31 TABLE 26 - ProblemEmbeddings

Purpose:
Stores embedding data for semantic similarity search.

Main fields:
- EmbeddingId BIGINT PK
- RequestId INT FK
- EmbeddingData VARBINARY(MAX)
- CreatedAt DATETIME2

Relationships:
- ProblemRequests 1:1 ProblemEmbeddings

Soft Delete:
NO by default.

Reason:
This is technical derivative data. Its lifecycle can be controlled by the owning request and search system without adding redundant deletion semantics.

32 TABLE 27 - SimilarityMatches

Purpose:
Stores similarity results between a current request and historical problems.

Main fields:
- MatchId BIGINT PK
- RequestId INT FK
- MatchedProblemId INT FK
- SimilarityScore DECIMAL(6,5)
- CreatedAt DATETIME2

Relationships:
- ProblemRequests 1:N SimilarityMatches
- Problems 1:N SimilarityMatches

Soft Delete:
NO by default.

Reason:
Similarity results are technical search results. They can be regenerated.
Normal similarity search must not use deleted Problems as active reusable knowledge unless an internal operation explicitly includes deleted data.

33 TABLE 28 - Plans

PURPOSE:
Stores the platform subscription/entitlement plans.

Example plans:
- FREE
- PRO
- PREMIUM

The current initial deployment should use:
- FREE as the default plan
- No payment gateway
- No paid charging

Main fields:

- PlanId INT PK
- Name NVARCHAR(100)
- Code VARCHAR(50) UNIQUE
- Description NVARCHAR(500) NULL
- Status VARCHAR(20)
- IsDefault BIT
- IsPublic BIT
- PeriodType VARCHAR(20)
- CreditsPerPeriod INT
- Price DECIMAL(18,2) NULL
- Currency VARCHAR(10) NULL
- CreatedAt DATETIME2
- UpdatedAt DATETIME2

Relationships:

- Plans 1:N PlanFeatures
- Plans 1:N UserSubscriptions
- Plans 1:N UsagePeriods

Soft Delete:
NO by default.

Reason:

Plans have a specific lifecycle:

- Draft
- Active
- Archived

Archiving a plan is preferable to deleting it.

Historical subscriptions and usage must continue referencing archived plans.

Default plan rule:
Exactly one active default plan should exist.

34 TABLE 29 - Features

PURPOSE:
Stores platform-level feature definitions.

Examples:

- AI_PROBLEM_ANALYSIS
- AI_FOLLOW_UP
- AI_ARTICLE_GENERATION
- SAVED_PROBLEM_HISTORY
- CHALLENGE_GENERATION
- CODE_REVIEW
- GITHUB_ANALYSIS

Main fields:

- FeatureId INT PK
- Key VARCHAR(100) UNIQUE
- Name NVARCHAR(100)
- Description NVARCHAR(500) NULL
- Category VARCHAR(50)
- CreatedAt DATETIME2
- UpdatedAt DATETIME2

Relationships:

- Features 1:N PlanFeatures

Soft Delete:
NO by default.

Reason:
Features are reference/configuration data.

35 TABLE 30 - PlanFeatures

PURPOSE:
Connects Plans to Features and stores feature-specific configuration.

Main fields:

- PlanId INT PK/FK
- FeatureId INT PK/FK
- IsEnabled BIT
- LimitValue INT NULL
- ConfigJson NVARCHAR(MAX) NULL
- CreatedAt DATETIME2
- UpdatedAt DATETIME2

Primary key:
(PlanId, FeatureId)

Relationships:

- Plans 1:N PlanFeatures
- Features 1:N PlanFeatures

Soft Delete:
NO by default.

Reason:
Feature configuration has explicit IsEnabled and limit semantics.

IMPORTANT:
Do not place every plan rule into an opaque JSON document. Core limits should remain strongly typed.

36 TABLE 31 - UserSubscriptions

PURPOSE:
Stores a user's plan assignment and its history.

Main fields:

- SubscriptionId BIGINT PK
- UserId INT FK
- PlanId INT FK
- Status VARCHAR(20)
- StartDate DATETIME2
- EndDate DATETIME2 NULL
- AssignedByUserId INT NULL
- AssignmentSource VARCHAR(30)
- CreatedAt DATETIME2
- UpdatedAt DATETIME2
- CancelledAt DATETIME2 NULL
- CancellationReason NVARCHAR(500) NULL

Relationships:

- Users 1:N UserSubscriptions
- Plans 1:N UserSubscriptions

Soft Delete:
NO.

Reason:
Subscription lifecycle is handled through Status and dates.

Examples:

- Active
- Cancelled
- Expired
- Scheduled

History:
Previous plans must remain stored.

NEW USER RULE:
New users receive the active default Free Plan automatically.
The frontend must never choose the plan during registration.

Existing users:
Existing users without a subscription require an explicit migration/backfill process rather than silently changing their subscription at login.

37 TABLE 32 - AIUsageEvents

PURPOSE:
Detailed provider-level usage accounting.
THIS IS THE MOST IMPORTANT ADDITION FOR TOKEN/COST TRACKING.

AIRequests answers:
"What logical AI operation did the application perform?"

AIUsageEvents answers:
"What actual provider/model call happened, and how much did it consume?"

Main fields:

- AIUsageEventId BIGINT PK
- AIRequestId BIGINT FK
- UserId INT FK
- ProblemId INT NULL FK
- ProviderId INT FK
- ModelId INT FK NULL
- OperationType VARCHAR(50)
- InputTokens INT NULL
- OutputTokens INT NULL
- TotalTokens INT NULL
- EstimatedCost DECIMAL(18,6) NULL
- ActualCost DECIMAL(18,6) NULL
- CostType VARCHAR(20)
- CreditsReserved INT NULL
- CreditsCharged INT NULL
- CreditsRefunded INT NULL
- Status VARCHAR(30)
- ProviderRequestId VARCHAR(200) NULL
- CorrelationId VARCHAR(100) NULL
- IdempotencyKey VARCHAR(150) NULL
- StartedAt DATETIME2
- CompletedAt DATETIME2 NULL
- DurationMs BIGINT NULL
- RetrySequence INT DEFAULT 0
- ErrorCode VARCHAR(100) NULL
- CreatedAt DATETIME2

Relationships:

- AIRequests 1:N AIUsageEvents
- Users 1:N AIUsageEvents
- Problems 1:N AIUsageEvents
- AIProviders 1:N AIUsageEvents
- AIModels 1:N AIUsageEvents

Soft Delete:
NO.

Reason:
Usage events are append-only accounting/operational records.

TOKEN RULE:

Prefer storing:

- InputTokens
- OutputTokens
- TotalTokens

TotalTokens should be:

- provider supplied total, OR
- correctly derived from input + output

COST RULE:

If provider pricing is unavailable:

- EstimatedCost may be stored
- ActualCost remains NULL
- CostType reflects the source

SECURITY:

Do not store API keys, provider secrets, raw passwords, or unnecessary sensitive prompt/response content in usage events.

RETRIES:

Every actual provider call can produce its own AIUsageEvent.

Example:

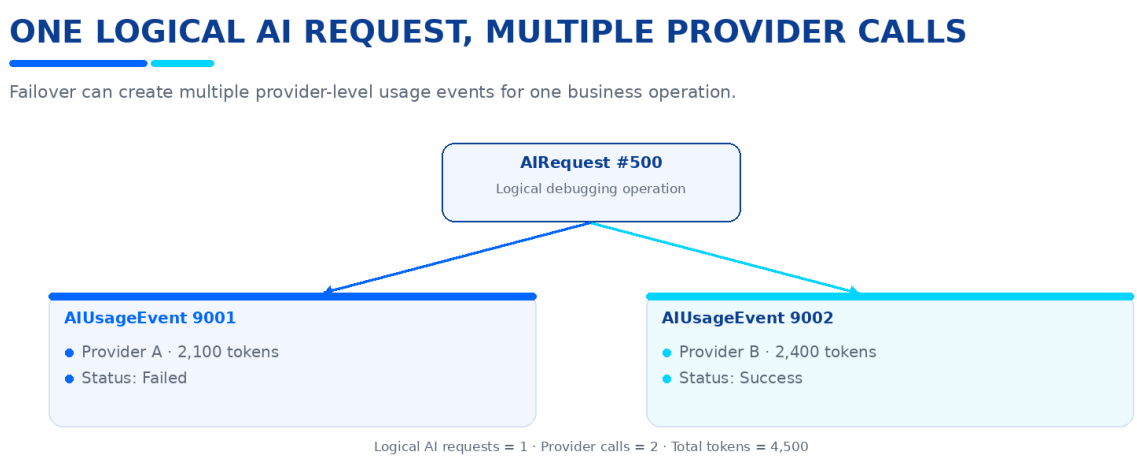


Figure. Provider-level usage during fallback.

Therefore:

Logical AI Requests = count(AIRequests)
Provider Calls = count(AIUsageEvents)
Total Tokens = sum(AIUsageEvents.TotalTokens)

38 TABLE 33 - UsagePeriods

PURPOSE:

Tracks cumulative usage within a quota period.

Main fields:

- UsagePeriodId BIGINT PK
- UserId INT FK
- PlanId INT FK
- PeriodStart DATETIME2
- PeriodEnd DATETIME2
- CreditsAllocated INT
- CreditsReserved INT
- CreditsUsed INT
- CreditsRefunded INT
- RequestCount BIGINT
- TokenCount BIGINT
- CreatedAt DATETIME2
- UpdatedAt DATETIME2

Relationships:

- Users 1:N UsagePeriods
- Plans 1:N UsagePeriods
- UsagePeriods 1:N UsageLedger entries

Soft Delete:

NO.

Reason:

This is historical quota/accounting state.

Purpose:

Answers:

- How much was allocated?
- How much was reserved?
- How much was used?
- How many requests were processed?
- How many tokens were consumed?

Usage period type follows the active Plan configuration.
Do not assume every future plan must be monthly.

39 TABLE 34 - UsageLedger

PURPOSE:
Provides an auditable credit movement ledger.

Main fields:

- UsageLedgerId BIGINT PK
- UserId INT FK
- UsagePeriodId BIGINT FK
- AIUsageEventId BIGINT NULL FK
- EntryType VARCHAR(30)
- Amount INT
- Reason NVARCHAR(200)
- ReferenceId VARCHAR(100) NULL
- MetadataJson NVARCHAR(MAX) NULL
- CreatedAt DATETIME2

Possible EntryType values:

- RESERVATION
- CHARGE
- RELEASE
- REFUND
- ADMIN_ADJUSTMENT

Relationships:

- Users 1:N UsageLedger
- UsagePeriods 1:N UsageLedger
- AIUsageEvents 1:N UsageLedger

Soft Delete:
NO.

Reason:
This is financial-style/accounting history even though no payment gateway is currently implemented.

Double-charge protection:
Each movement must have a unique reference/idempotency strategy so a retry cannot apply the same credit movement twice.

40 TABLE 35 - AuditLogs

PURPOSE:
Stores security-sensitive and administrative actions.

Typical events:

- LOGIN_SUCCESS
- LOGIN_FAILED
- ACCOUNT_CREATED
- EMAIL_VERIFIED
- PASSWORD_RESET
- LOGOUT
- REFRESH_TOKEN_REUSE
- ACCOUNT_SOFT_DELETED
- PLAN_CREATED
- PLAN_UPDATED
- PLAN_ARCHIVED
- DEFAULT_PLAN_CHANGED

- USER_PLAN_CHANGED
- CREDIT_ADJUSTED
- SECURITY_FAILURE

Main fields:

- AuditLogId BIGINT PK
- UserId INT NULL FK
- Action VARCHAR(100)
- EntityName VARCHAR(100)
- EntityId VARCHAR(100) NULL
- OldValues NVARCHAR(MAX) NULL
- NewValues NVARCHAR(MAX) NULL
- IpAddress VARCHAR(50) NULL
- UserAgent NVARCHAR(500) NULL
- RequestId VARCHAR(100) NULL
- CreatedAt DATETIME2

Relationships:

- Users 1:N AuditLogs

Soft Delete:
NO.

Reason:
Audit logs are append-only security history.

Security:

NEVER store:

- passwords
- password hashes
- raw access tokens
- raw refresh tokens
- OTP codes
- password-reset tokens
- API keys
- provider secrets

The audit record should contain safe metadata and a concise change summary.

41 TABLE 36 - RateLimitLogs (OPTIONAL)

PURPOSE:
Optional durable telemetry for rate-limit rejection/behavior.

IMPORTANT:
This table is NOT required by the native ASP.NET Core limiter.
Native rate limiting remains the actual enforcement mechanism.

Suggested fields:

- RateLimitLogId BIGINT PK
- UserId INT NULL FK
- IpAddress VARCHAR(50) NULL
- Endpoint VARCHAR(200)
- PolicyName VARCHAR(100)
- Allowed BIT
- Reason VARCHAR(200) NULL
- RequestId VARCHAR(100) NULL
- CreatedAt DATETIME2

Soft Delete:
NO.

Recommendation:
Do not persist every successful request here.

Prefer recording:

- rejected requests
- important throttling events
- aggregated metrics

Avoid creating a high-volume table if application/host metrics already provide the required monitoring.

42 PLAN MANAGEMENT RULES

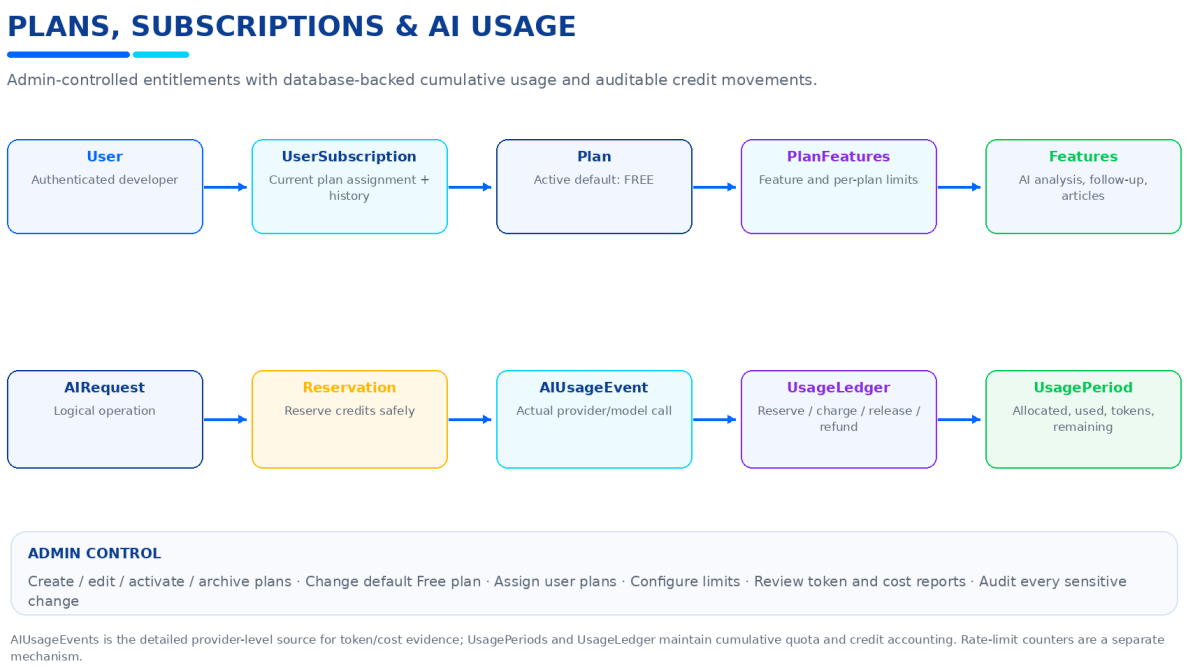


Figure. Plans, subscriptions, feature entitlements, credits and usage accounting.

Initial default:

FREE

New-user registration:

Client sends:

Name
Email
Password
Turnstile
Client does NOT send PlanId.

Backend:

1. Create User
2. Load active default Plan
3. Create UserSubscription
4. Continue email verification

The assignment must happen transactionally.

Concurrency requirement:

Two simultaneous registrations must not create an invalid duplicate default subscription state.

Default-plan rule:

Exactly one active default plan must exist.

Plan archive:

Archived plans:

- remain in the database
- remain referenced by historical subscriptions
- cannot receive prohibited new assignments

Plan deletion:

Do NOT physically delete plans.

Plan limit changes:

The initial policy is:

- administrator changes apply to current subscribers
- historical usage remains unchanged
- historical subscription records remain unchanged

43 FEATURE ENTITLEMENT RULES

Plan feature access must be checked server-side.

Example:

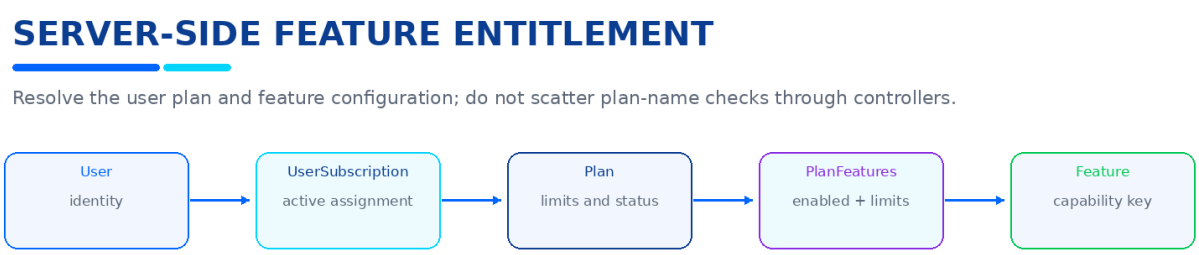


Figure. Server-side feature entitlement lookup.

Do NOT write:

```
if (plan == "FREE") ...
```

throughout controllers/services.

Use the plan/feature model as the source of truth.

Example feature:

AI_PROBLEM_ANALYSIS

Example:

Free:

Enabled = true

LimitValue = 50 credits

Future Pro:

Enabled = true

LimitValue = 500 credits

The exact commercial limits are configuration data, not application logic.

44 AI QUOTA AND CREDIT RULES

AI Usage Quota controls cumulative consumption.

A request must pass all of the following:

1. Authentication
2. Authorization
3. Rate limit
4. Input validation
5. Plan feature access
6. Available quota/credits
7. Concurrency protection

Reservation:

Before expensive AI execution:

Reserve credits

Provider call:

Execute AI request

Finalization:

Capture actual tokens

Capture actual/estimated cost

Charge actual usage

Release unused reservation

Example:

Reserved = 5 credits

Actual = 3 credits

Charge = 3

Release = 2

Failure:

If the provider definitely did not process the request:

Release reservation

Ambiguous timeout:

If the provider may have processed the request:

Do NOT automatically refund and retry blindly.

Reconcile according to provider behavior and idempotency strategy.

45 AI REQUEST VS AI USAGE EVENT

This distinction is mandatory.

AIRequest:

Business/logical AI operation.

AIUsageEvent:

Actual provider/model execution.

Example:

User requests:

"Debug JWT 401 problem"

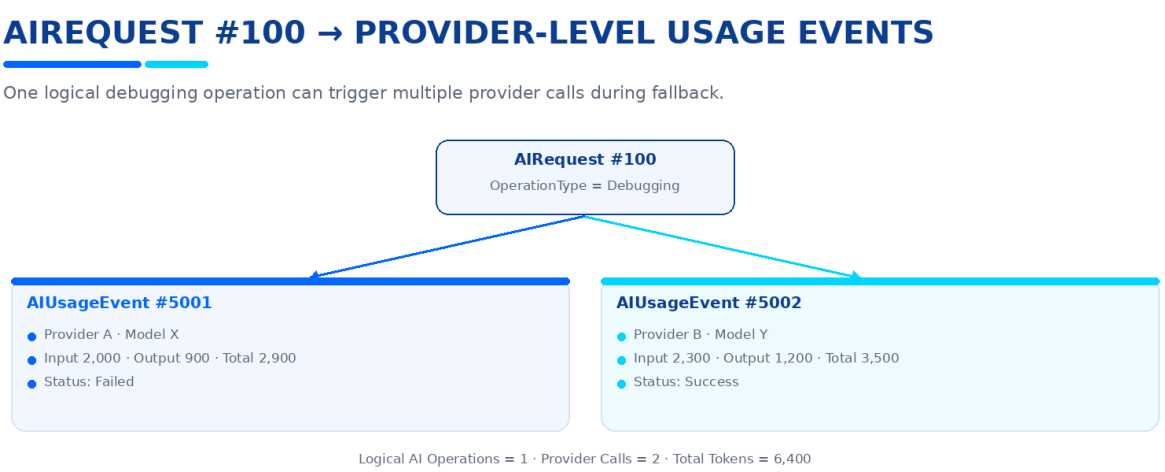


Figure. One logical AIRequest can create multiple AIUsageEvents.

Result:

Logical AI Operations: 1

Provider Calls: 2

Total Tokens: 6400

This gives the admin dashboard accurate provider-level accounting.

46 AI OPERATION TYPES

Recommended operation types for DevPilot AI:

- ProblemAnalysis
- FollowUp
- RootCauseAnalysis
- ArticleGeneration
- ChallengeGeneration
- ChallengeEvaluation
- CodeReview
- RepositoryAnalysis
- SolutionSynthesis
- TelegramKnowledgeSearch
- Other future AI operations

Operation types must be controlled values.

Do not allow the frontend to submit arbitrary usage classifications for billing/credit purposes.

47 TOKEN ACCOUNTING

For every provider call where data is available, record:
InputTokens
OutputTokens
TotalTokens

Example:

InputTokens = 3200
OutputTokens = 1800
TotalTokens = 5000

If the provider supplies only total tokens:

Store TotalTokens
Leave unavailable component fields NULL.

Token counts must come from the provider response or trusted server-side calculation.
NEVER trust token counts supplied by the frontend.

Admin reporting can calculate:

Total Tokens
Average Tokens / Request
Tokens / Operation
Tokens / Provider
Tokens / Model
Tokens / User
Tokens / Plan

48 COST ACCOUNTING

Store:
EstimatedCost
ActualCost
CostType

Example:

EstimatedCost = 0.002100
ActualCost = 0.001800
CostType = Actual

Or:

EstimatedCost = 0.002100
ActualCost = NULL
CostType = Estimated

This makes the system ready for future paid providers without changing the architecture.
Current free-provider usage can still be tracked even when actual cost is zero.

49 RATE LIMITING ARCHITECTURE

Use ASP.NET Core native rate limiting:

Microsoft.AspNetCore.RateLimiting

Central components:

- AddRateLimiter
- UseRateLimiter
- Named policies
- Partitioned limits
- OnRejected

Recommended policies:

GENERAL_API

Example:

100 requests/minute/user

AI_ANALYSIS

Example:

5 requests/minute/user

AI_FOLLOW_UP

Example:

10 requests/minute/user

AI_ARTICLE

Example:

3 requests/minute/user

AUTH_LOGIN

Example:

5 attempts/5 minutes/client

AUTH_REGISTER

Example:

5 attempts/5 minutes/client

AI_CONCURRENCY

Example:

1 concurrent expensive AI operation/user

These are development defaults only.

Final production values should be configurable.

50 RATE LIMIT PARTITIONING

Authenticated endpoints:

Partition by trusted UserId.

Anonymous endpoints:

Partition by validated client/IP identity where appropriate.

Do not trust arbitrary client-provided headers as partition keys.

If deployed behind a reverse proxy:

Configure trusted forwarded headers correctly.

Do not blindly trust X-Forwarded-For from untrusted sources.

Authentication endpoints require both:

- client/IP protection
- account-level protections where appropriate

This prevents attackers from evading limits by switching target accounts.

51 RATE LIMIT RESPONSE

Exceeded limit:

HTTP 429 Too Many Requests

Standard error envelope:

```
{
"success": false,
"errorCode": "RATE_LIMIT_EXCEEDED",
```

```
"message": "Too many requests. Please try again later.",
"requestId": "..."
}
```

Where supported:

Retry-After: <seconds>

Rate-limited requests MUST be rejected before invoking the LLM provider.

Rate-limit configuration details must not be exposed unnecessarily.

52 RATE LIMIT VS AI QUOTA

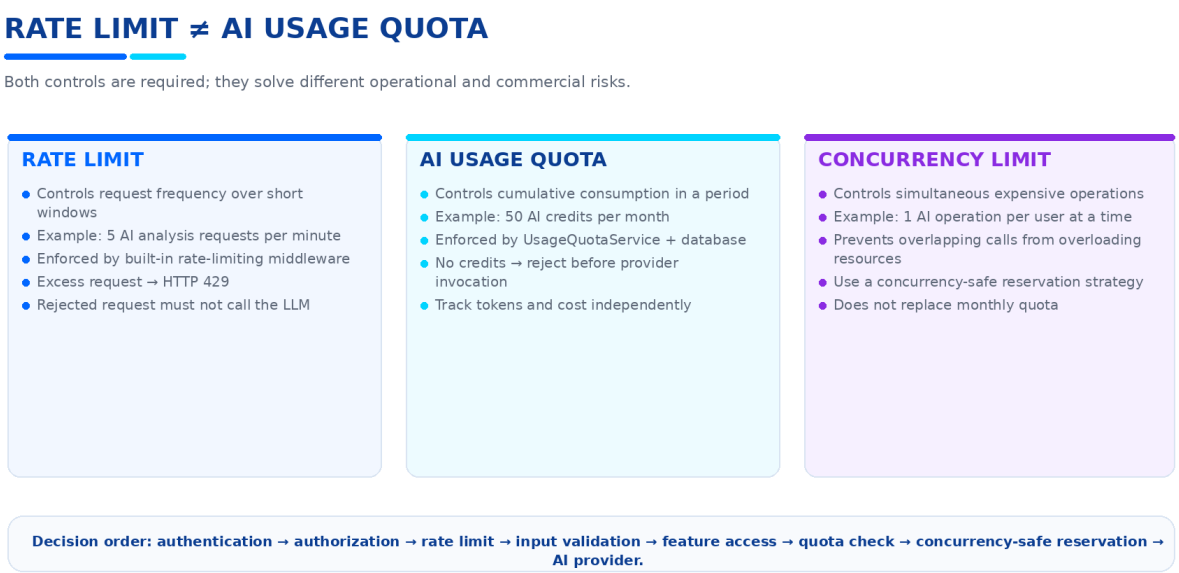


Figure. Request Rate Limiting, AI Usage Quota, and Concurrency Limit are distinct controls.

Example:

User Plan:

50 AI Credits / period

Rate Limit:

5 AI analysis requests / minute

Scenario A:

User makes 5 requests in one minute.

Rate limit allows them.

Quota is still reduced according to actual usage.

Scenario B:

User has 0 credits remaining.

Rate limit allows the request.

Quota service rejects it BEFORE provider invocation.

Scenario C:

User sends 20 requests in one second.

Rate limiter rejects excess requests with 429.

Those rejected requests consume no AI credits.

Therefore:

Rate Limit = request frequency protection

Quota = cumulative consumption protection

53 AI OPERATION END-TO-END FLOW

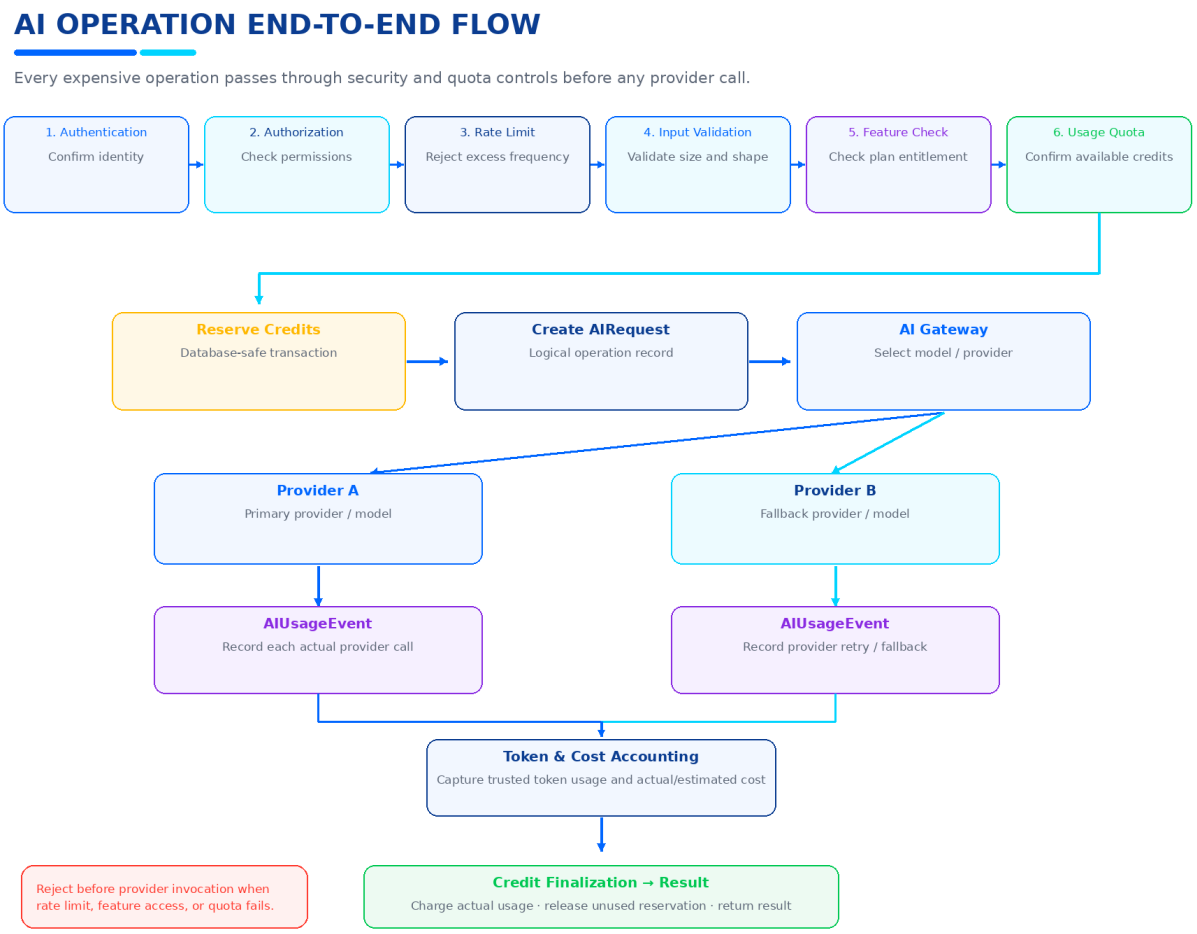


Figure. Secure AI operation processing and accounting flow.

54 CONCURRENCY CONTROL

AI credit accounting must be concurrency-safe.

Example:

User balance = 10 credits
Request A wants 8
Request B wants 8

Incorrect:

A reads 10
B reads 10
A spends 8
B spends 8

Correct:

A reserves 8
remaining = 2
B attempts to reserve 8
B is rejected

Reservation and accounting must use database-safe transactions/locking or an equivalent concurrency-safe mechanism.

Do not rely on an in-memory variable as the durable credit balance.

55 IDEMPOTENCY

Idempotency is important for:

- AI usage charging

- Credit reservation
- Credit release
- Provider retries
- Administrative credit adjustments

Recommended fields:

- IdempotencyKey
- ProviderRequestId
- ReferenceId

The same operation must not produce double charging because of:

- HTTP retries
- provider retry
- client timeout
- network retry
- worker retry

56 SOFT DELETE ARCHITECTURE

Soft-deletable entities:

1. Users
2. Problems
3. ProblemAttachments
4. ProblemSolutions
5. KnowledgeArticles
6. Challenges
7. GitHubRepositories
8. CodeReviews

Common fields:

IsDeleted BIT NOT NULL DEFAULT 0

DeletedAt DATETIME2 NULL

Global query filter:

entity.IsDeleted == false

Normal business queries:

Hide deleted rows automatically.

Internal/admin:

May intentionally include deleted rows.

Do NOT apply soft delete automatically to:

- Roles
- UserRoles
- EmailVerifications
- RefreshTokens
- ProblemRequests
- DebuggingAttempts
- DeveloperFeedback
- ArticleVersions
- ChallengeAttempts
- ChallengeEvaluations
- GitHubAccounts
- CodeReviewFindings
- AIProviders
- AIModels
- AIRequests
- ProblemEmbeddings
- SimilarityMatches
- Plans
- Features
- PlanFeatures
- UserSubscriptions
- AIUsageEvents
- UsagePeriods
- UsageLedger
- AuditLogs

57 USER ACCOUNT SOFT DELETE FLOW

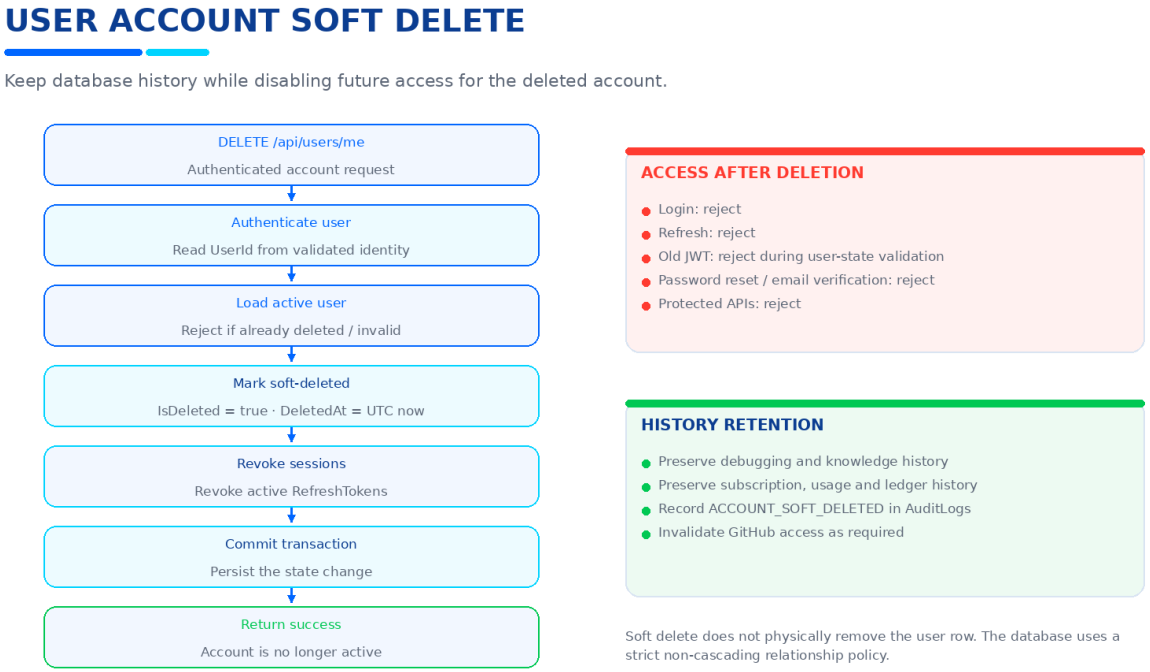


Figure. User account soft-delete flow and access outcomes.

58 KNOWLEDGE RETENTION AFTER ACCOUNT DELETION

User deletion does NOT automatically mean knowledge deletion.

Preserve where required:

- Solved Problems
- Problem Solutions
- Knowledge Articles
- Article Versions
- Debugging history

Public presentation must follow privacy rules.

The author can later be represented as:

Deleted Developer

or another approved anonymous representation.

Never expose:

- private email
- password data
- GitHub credentials
- private repository content

The exact public-author policy must be kept consistent across APIs and frontend.

59 QUERY FILTER RULES

Normal application queries must not expose Soft Deleted entities.

Examples:

- Problem list
- Problem detail
- Challenge list
- Knowledge Base
- Code Review dashboard
- GitHub repository workspace
- User current profile

Search/AI reuse must also respect soft-delete state.

Similarity search must not treat deleted Problems as active reusable knowledge.

AI context builders must not include deleted private/user-owned content unless

an authorized internal operation intentionally requires it.

60 UNIQUE INDEX STRATEGY

Important unique keys:

Users.Email
Problems.ProblemCode
Roles.Name
Technologies.Name
GitHubRepositories.ExternalRepositoryId
Plans.Code
Features.Key

Do not automatically convert every unique index into a filtered index.
Decide per business rule.

Example:

Users.Email should normally remain reserved by the deleted account unless
a future explicit product requirement allows identity recycling.
Plans.Code must remain unique to prevent ambiguous configuration.

61 TRANSACTIONAL OPERATIONS

Registration:

- Create User
- Assign default Free Plan
- Create UserSubscription
- Create EmailVerification

These operations should be coordinated safely.

Account deletion:

- Soft-delete User
- Revoke active RefreshTokens

Both must complete atomically.

AI execution:

- Reserve credits
- Create usage record
- Process provider call
- Finalize charge/release

The accounting flow must protect against races and duplicate charging.

Plan change:

- End current subscription if required
- Create new subscription
- Preserve history
- Audit the change

Default plan change:

- Update previous default
- Set new default
- Ensure the system never ends with zero active defaults or two active defaults.

62 ADMIN CAPABILITIES

Admin should eventually be able to:

PLAN:

- Create Plan
- List Plans
- Get Plan
- Update Plan
- Activate Plan
- Archive Plan
- Change Default Plan

SUBSCRIPTIONS:

- View user subscription

- Assign plan
- Change user plan
- View subscription history

USAGE:

- View total AI requests
- View provider calls
- View tokens
- View costs
- View credits
- View quota usage
- View provider/model statistics
- View failures
- View rate-limit rejections

SECURITY:

- Review AuditLogs
- Review administrative actions
- Review security failures

63 USER USAGE CAPABILITIES

A Developer should be able to read:

- Current Plan
- Subscription Status
- Available Features
- Current Usage Period
- Credits Allocated
- Credits Used
- Credits Remaining
- AI Requests Count
- Token Consumption
- Relevant usage history

A Developer must NOT be able to:

- change PlanId arbitrarily
- increase credits
- modify token counts
- modify usage events
- modify subscription status
- retrieve another user's usage data

64 USAGE ANALYTICS

Recommended Admin metrics:

GENERAL:

- Total AI logical requests
- Total provider calls
- Total input tokens
- Total output tokens
- Total tokens
- Estimated cost
- Actual cost

BY PROVIDER:

- Requests
- Tokens
- Failures
- Average latency
- Success rate

BY MODEL:

- Requests
- Tokens
- Average tokens/request
- Availability
- Failures

BY OPERATION:

- Problem Analysis
- Follow-up
- Article Generation
- Challenge Generation
- Challenge Evaluation
- Code Review
- Repository Analysis
- Solution Synthesis

BY USER:

- Requests
- Provider calls
- Tokens
- Credits used
- Cost

BY PLAN:

- Active subscriptions
- Allocated credits
- Used credits
- Average consumption

65 AI GATEWAY RELATIONSHIP WITH ACCOUNTING

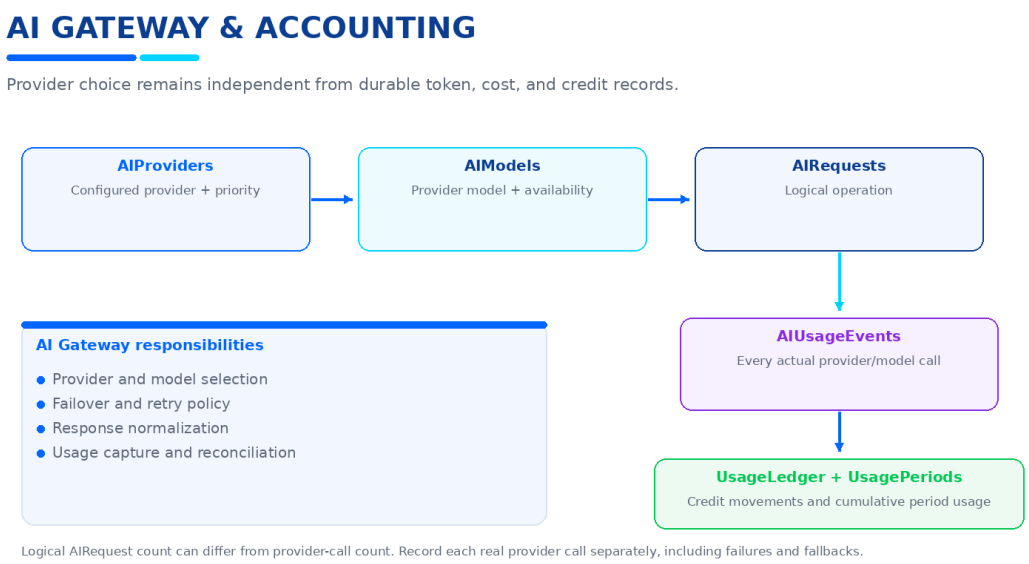


Figure. AI Gateway relationship with durable accounting.

AI Gateway responsibilities:

- provider selection
- model selection
- failover
- retry policy
- response normalization
- usage capture
- usage reconciliation

66 TOKEN OPTIMIZATION DATA FLOW

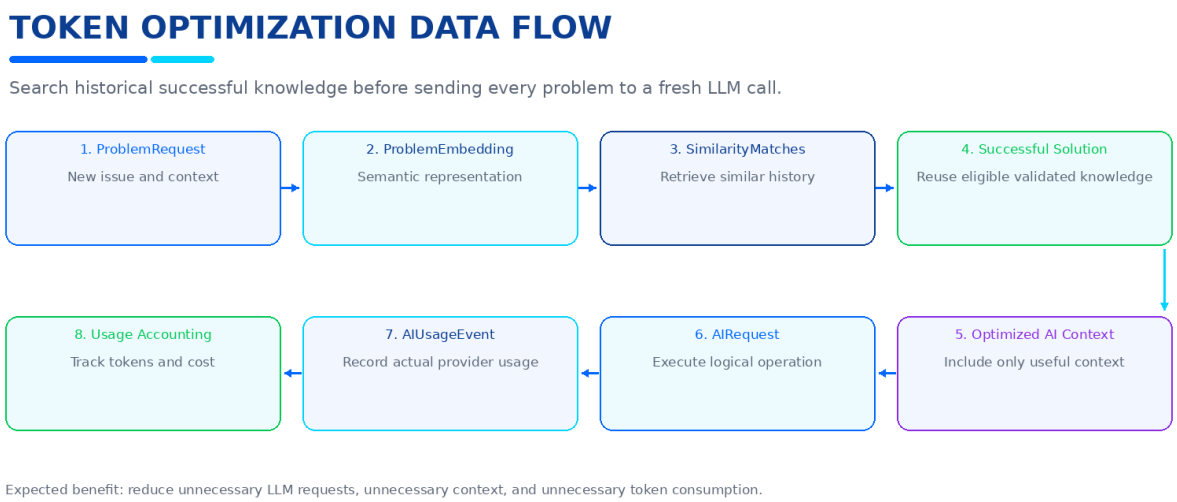


Figure. Token optimization data flow.

The database supports token optimization by preserving successful reusable knowledge before sending every problem to a fresh LLM call.

The goal is to reduce:

- unnecessary LLM requests
- unnecessary context
- unnecessary token consumption

67 GITHUB AND AI USAGE

GitHub operations that call AI should still pass through:

- Authentication
- Authorization
- Rate limiting
- Plan feature checks
- Usage quota checks
- Usage reservation
- AI Gateway
- AIUsageEvents

Examples:

- Repository analysis
- GitHub-based debugging
- Code review
- Challenge evaluation

68 TELEGRAM AND AI USAGE

Telegram flow:

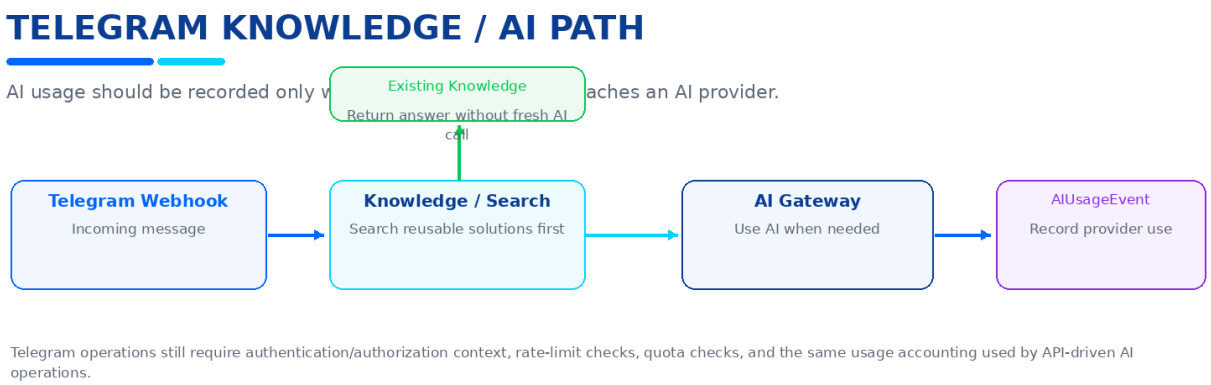


Figure. Telegram knowledge-first AI flow.

Telegram may therefore create AIUsageEvents when AI is actually used.

69 APPEND-ONLY DATA

The following should be treated as historical/append-only or lifecycle-based:

- ProblemRequests
- DebuggingAttempts
- DeveloperFeedback
- ArticleVersions
- ChallengeAttempts
- ChallengeEvaluations
- CodeReviewFindings
- AIRequests
- AIUsageEvents
- UsageLedger
- AuditLogs

They should not be physically deleted as part of ordinary user operations.

70 PLAN / USAGE SOFT DELETE RULE

Plans:
Archive, do not delete.

Features:
Maintain as reference data.

PlanFeatures:
Use IsEnabled and configuration.

UserSubscriptions:
Use status and dates.

AIUsageEvents:
Preserve accounting history.

UsagePeriods:
Preserve historical periods.

UsageLedger:
Preserve every accounting movement.

AuditLogs:
Preserve security history.

This prevents "Soft Delete everywhere" from becoming technically misleading.

71 DATABASE INDEXING RECOMMENDATIONS

High-value indexes:

Users:

- Email UNIQUE

Problems:

- ProblemCode UNIQUE
- UserId
- Status
- CreatedAt
- IsDeleted where supported by query pattern

ProblemRequests:

- ProblemId
- CreatedAt

DebuggingAttempts:

- ProblemId
- RequestId
- AttemptNumber

ProblemSolutions:

- ProblemId
- IsPreferred
- IsDeleted

KnowledgeArticles:

- ProblemId UNIQUE
- Status
- IsDeleted

Challenges:

- ProblemId
- Difficulty
- IsDeleted

GitHubRepositories:

- GitHubAccountId
- ExternalRepositoryId
- IsDeleted

CodeReviews:

- UserId
- GitHubRepositoryId
- Status
- IsDeleted

AIRequests:

- UserId
- ProblemId
- ModelId
- OperationType
- CreatedAt

AIUsageEvents:

- UserId
- AIRequestId
- ProviderId
- ModelId
- OperationType
- CreatedAt
- IdempotencyKey UNIQUE where appropriate

UsagePeriods:

- UserId + PeriodStart + PeriodEnd
- PlanId

UsageLedger:

- UserId
- UsagePeriodId

- AIUsageEventId
- ReferenceId / idempotency reference

Plans:

- Code UNIQUE
- IsDefault / Status combinations as required by SQL Server constraints

Features:

- Key UNIQUE

72 DATA INTEGRITY RULES

Never trust the frontend for:

- PlanId
- IsAdmin
- Credit balance
- Token counts
- Cost
- Subscription status
- Usage values
- Rate-limit partition identity

All important values must be determined server-side.

Never allow a normal Developer endpoint to mutate:

- AIUsageEvents
- UsageLedger
- AuditLogs
- Plan definitions
- UserSubscriptions outside allowed user-facing operations

73 SECURITY RULES

The database must never contain plaintext secrets.

Never store:

- Brevo API keys
- LLM provider keys
- GitHub access tokens in plaintext
- Passwords
- raw refresh tokens
- raw OTP codes
- reset tokens

Store:

- hashes
- encrypted credentials where required
- secure references
- safe metadata

Audit and usage records must not accidentally capture sensitive secrets from prompts, headers, or exceptions.

74 ERROR AND FAILURE ACCOUNTING

AI operation statuses should distinguish:

- Pending
- Success
- Failed
- RateLimited
- Cancelled

Provider-level usage event may additionally track:

- Provider timeout
- Provider rejection
- Validation failure
- Fallback success
- Unknown/ambiguous outcome

Quota behavior:

REJECTED BEFORE AI:
No AI usage charge.

PROVIDER DEFINITELY NOT CALLED:
Reservation can be released.

PROVIDER CALLED AND FAILED:
Preserve the usage event and follow the configured accounting policy.

TIMEOUT WITH UNKNOWN PROVIDER OUTCOME:
Do not blindly refund and retry.

75 ADMIN PLAN CHANGE AUDITING

Every important plan operation should create an AuditLog.

Example:

Action:
PLAN_LIMIT_UPDATED

Entity:
Plan

EntityId:
2

OldValues:
CreditsPerPeriod = 50

NewValues:
CreditsPerPeriod = 100

Actor:
Administrator UserId

CreatedAt:
UTC timestamp

This gives administrators accountability without storing secrets.

76 FUTURE PAYMENT SUPPORT

Payment is intentionally NOT part of the current implementation.

The current architecture only prepares for:

- Plan definitions
- Subscription history
- Limits
- Credit allocation
- Usage accounting
- Audit

Future payment entities may later include:

- Invoices
- Payments
- PaymentProviders
- ExternalCustomerReferences
- SubscriptionBillingEvents

Do not add those now unless the payment phase begins.

77 FUTURE DISTRIBUTED RATE LIMITING

Initial deployment:

- Native ASP.NET Core rate limiter
- Single backend instance
- In-process limiter state is acceptable

Future multi-instance deployment:
Re-evaluate distributed coordination.

IMPORTANT:
Durable AI quota accounting always remains in SQL Server.

Redis should NOT be introduced merely because a rate limiter exists.

78 TABLE LIFECYCLE CLASSIFICATION SUMMARY

SOFT DELETABLE:

- Users
- Problems
- ProblemAttachments
- ProblemSolutions
- KnowledgeArticles
- Challenges
- GitHubRepositories
- CodeReviews

LIFECYCLE / STATUS BASED:

- EmailVerifications
- RefreshTokens
- GitHubAccounts
- AIProviders
- AIModels
- Plans
- PlanFeatures
- UserSubscriptions

APPEND-ONLY / HISTORICAL:

- ProblemRequests
- DebuggingAttempts
- DeveloperFeedback
- ArticleVersions
- ChallengeAttempts
- ChallengeEvaluations
- CodeReviewFindings
- AIRequests
- AIUsageEvents
- UsageLedger
- AuditLogs

REFERENCE / SYSTEM:

- Roles
- Technologies
- Features

TECHNICAL / DERIVED:

- ProblemEmbeddings
- SimilarityMatches
- UsagePeriods
- Optional RateLimitLogs

JOIN / RELATIONSHIP:

- UserRoles
- ProblemTechnologies

79 MASTER RELATIONSHIP MAP

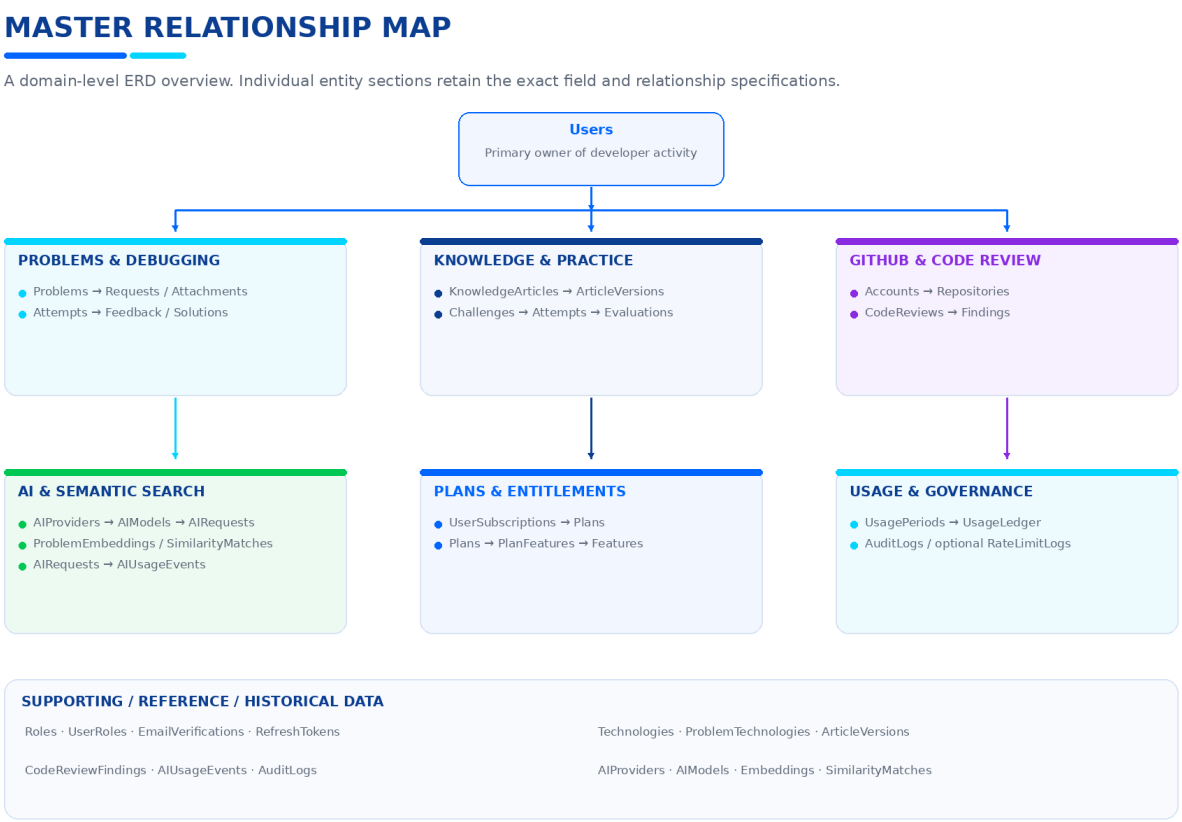


Figure. Master relationship map by business domain.

Relationship highlights

- Users → UserRoles → Roles; EmailVerifications; RefreshTokens; Problems; GitHubAccounts; AIRequests; UserSubscriptions → Plans; UsagePeriods → UsageLedger; AuditLogs.
- Problems → ProblemRequests → ProblemAttachments, ProblemEmbeddings, SimilarityMatches, and DebuggingAttempts; Problems → DebuggingAttempts → DeveloperFeedback and ProblemSolutions.
- Problems → KnowledgeArticles → ArticleVersions; ProblemTechnologies → Technologies; Challenges → ChallengeAttempts → ChallengeEvaluations.
- GitHubAccounts → GitHubRepositories → CodeReviews → CodeReviewFindings.
- AIProviders → AIModels → AIRequests and AIUsageEvents; AIRequests → AIUsageEvents.
- Plans → PlanFeatures → Features; UserSubscriptions → Plans; UsagePeriods → UsageLedger; ProblemRequests → ProblemEmbeddings and SimilarityMatches; Problems → SimilarityMatches.

80 COMPLETE BUSINESS FLOW WITH THE UPDATED MODEL

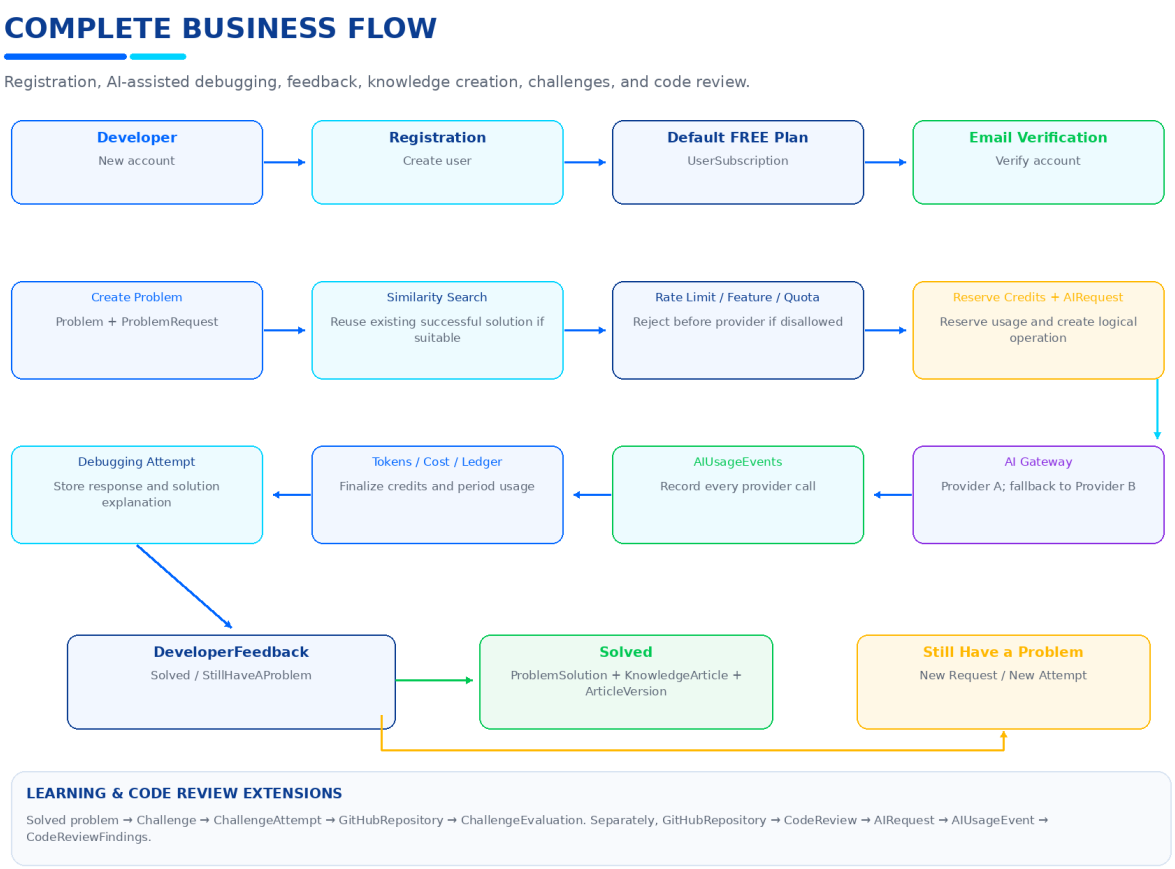


Figure. Complete DevPilot AI business flow.

81 ACCOUNT DELETION WITH THE UPDATED PLAN/USAGE MODEL

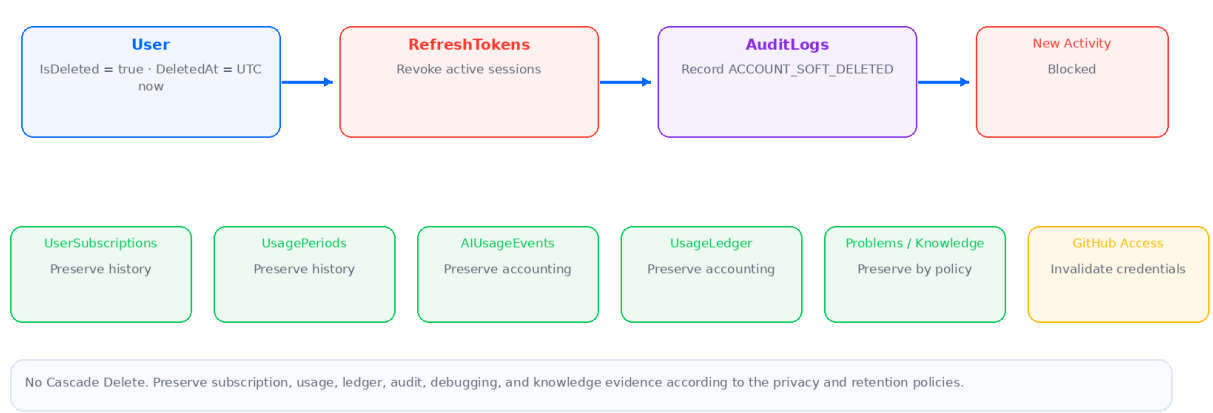


Figure. Account deletion with subscription and usage history retention.

When a User deletes the account:

User:

IsDeleted = true

DeletedAt = UTC now

RefreshTokens:
Revoke active sessions

UserSubscriptions:
Preserve history

UsagePeriods:
Preserve history

AIUsageEvents:
Preserve accounting history

UsageLedger:
Preserve accounting history

AuditLogs:
Record ACCOUNT_SOFT_DELETED

Problems:
Preserve by default

Knowledge:
Preserve according to privacy/publication policy

GitHub:
Credentials/access must be invalidated as required

New activity:
Blocked

82 DO NOT CONFUSE THESE CONCEPTS

Soft Delete:
"This business object is no longer active/visible."

Archive:
"This configuration/content is retired but remains a valid historical record."

Revoke:
"This credential/session is no longer valid."

IsActive:
"This configuration/provider is currently enabled."

IsAvailable:
"This model/service is currently usable."

IsUsed:
"This refresh token has already been consumed."

Quota:
"How much cumulative AI consumption is allowed."

Rate Limit:
"How frequently requests may arrive."

AIRequest:
"One logical AI operation."

AIUsageEvent:
"One actual provider/model execution."

83 IMPLEMENTATION ORDER FOR THE NEW SUBSYSTEM

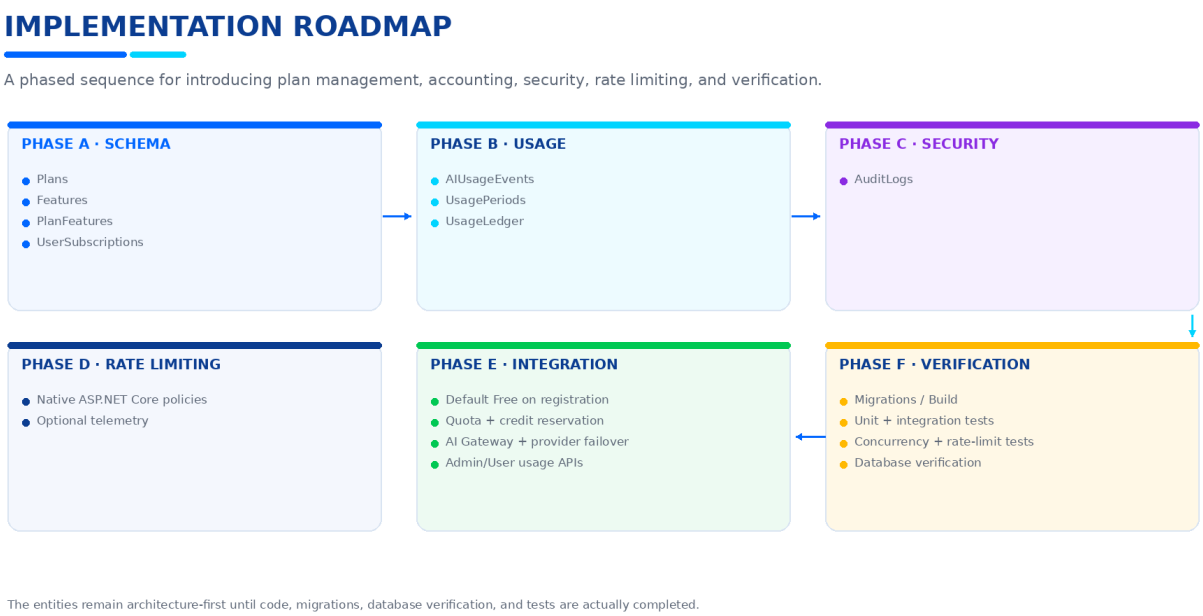


Figure. Implementation roadmap for the plan and usage subsystem.

Recommended sequence:

PHASE A - Schema:

1. Plans
2. Features
3. PlanFeatures
4. UserSubscriptions

PHASE B - Usage Accounting:

5. AIUsageEvents
6. UsagePeriods
7. UsageLedger

PHASE C - Security:

8. AuditLogs

PHASE D - Rate Limiting:

9. Native ASP.NET Core rate-limiting policies
10. Optional RateLimitLogs/telemetry

PHASE E - Integration:

11. Assign default Free Plan during registration
12. Add quota service
13. Add credit reservation/finalization
14. Integrate AIUsageEvents with AI Gateway
15. Integrate usage with provider failover
16. Add Admin usage APIs
17. Add User usage APIs

PHASE F - Verification:

18. Migrations
19. Build
20. Unit tests
21. Integration tests
22. Concurrency tests
23. Rate-limit tests
24. Database verification

84 EXPECTED DATABASE SAFETY TESTS

Test:

- Existing data remains after new migrations.
- Existing users become linked safely to Free Plan where required.
- Exactly one active default plan exists.
- Duplicate default plans are prevented.
- Duplicate credit charges are prevented.
- Concurrent credit reservations are safe.
- No Cascade Delete exists.
- Soft-deleted rows remain in SQL Server.
- Normal queries hide soft-deleted rows.
- Internal queries can intentionally retrieve deleted rows.
- Deleted users cannot authenticate.
- Deleted users cannot use old JWTs.
- Deleted users cannot refresh.
- Deleted users cannot trigger password recovery.
- Deleted users cannot verify email.
- Rate-limited AI calls do not invoke the provider.
- Quota-exceeded AI calls do not invoke the provider.
- Provider calls create AIUsageEvents.
- Input/output/total tokens are captured correctly.
- Estimated and actual costs are distinguished.
- Usage ledger cannot double-charge.
- Audit records do not contain secrets.

85 FINAL ARCHITECTURAL DECISION

The updated DevPilot AI architecture is intentionally divided into:

CORE PRODUCT DATA
- Users
- Problems
- Knowledge
- Challenges
- GitHub
- Code Review

AUTHENTICATION / SECURITY
- Roles
- UserRoles
- EmailVerifications
- RefreshTokens
- AuditLogs

COMMERCIAL / ENTITLEMENT FOUNDATION
- Plans
- Features
- PlanFeatures
- UserSubscriptions

AI ACCOUNTING
- AIProviders
- AIModels
- AIRequests
- AIUsageEvents
- UsagePeriods
- UsageLedger

SEARCH / OPTIMIZATION
- ProblemEmbeddings
- SimilarityMatches
- ProblemSolutions

PROTECTION
- Native Rate Limiting

AI Concurrency Limits

SOFT DELETE

Only meaningful business entities receive:

IsDeleted

DeletedAt

NO CASCADE

All foreign keys remain non-cascading.

The database therefore becomes capable of answering:

"Who used the system?"
"What plan are they on?"
"What features are enabled?"
"How many logical AI requests were made?"
"How many provider calls actually happened?"
"Which provider/model handled them?"
"How many input tokens were used?"
"How many output tokens were used?"
"How many total tokens were consumed?"
"What was the estimated/actual cost?"
"How many credits were reserved?"
"How many credits were charged?"
"How many credits were refunded?"
"How much of the user's period quota remains?"
"Which requests were rejected by rate limiting?"
"Which requests failed?"
"What changed administratively?"
"What was soft-deleted?"
"Can historical knowledge still be preserved?"

This is the intended data foundation for future Plan Management, AI Usage, Credits, Quotas, Rate Limiting, and Administrative Monitoring without redesigning the core DevPilot AI database.

86 IMPORTANT IMPLEMENTATION NOTES

1. The new Plan/Usage entities are architecture-first until implemented and migrated in the actual repository.
2. Do not mark Jira tasks complete just because these tables are documented.
3. Do not introduce a payment gateway in the current phase.
4. Do not introduce Redis solely for rate limiting.
5. Do not use in-memory counters as the durable AI credit source of truth.
6. Do not allow frontend clients to choose privileged plans or modify usage.
7. Do not store secrets in usage or audit records.
8. Do not use Cascade Delete.
9. Do not add Soft Delete to entities that already have a more accurate lifecycle model.
10. Preserve historical debugging, knowledge, usage, and audit evidence.

END OF DOCUMENT